

B. Data Structures, Symmetries, and Neural Network Architectures

B.1 Tabular data: MLPs, residual connections, and autoencoders

B.2 Grid-structured data: CNNs

B.3 Sequential data: RNNs, LSTMs, and GRUs

B.4 Graph-structured data: GNNs and message passing

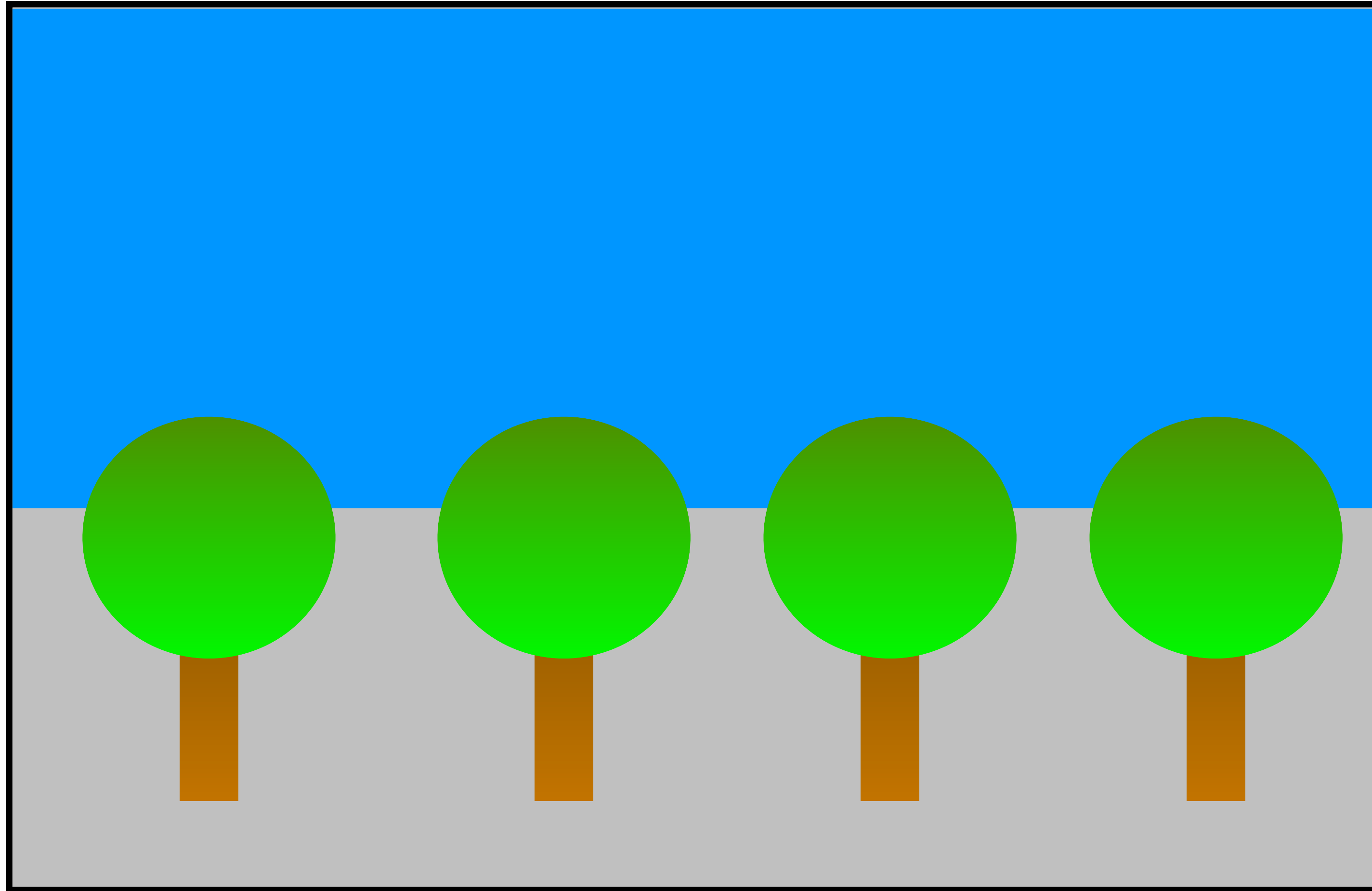
B.5 Set-structured data: Deep Sets



Tiam Heydari
Postdoctoral Fellow, UBC

Convolutional Neural Netwrks (CNNs): Basics and introduction

- How we (human) understand images?



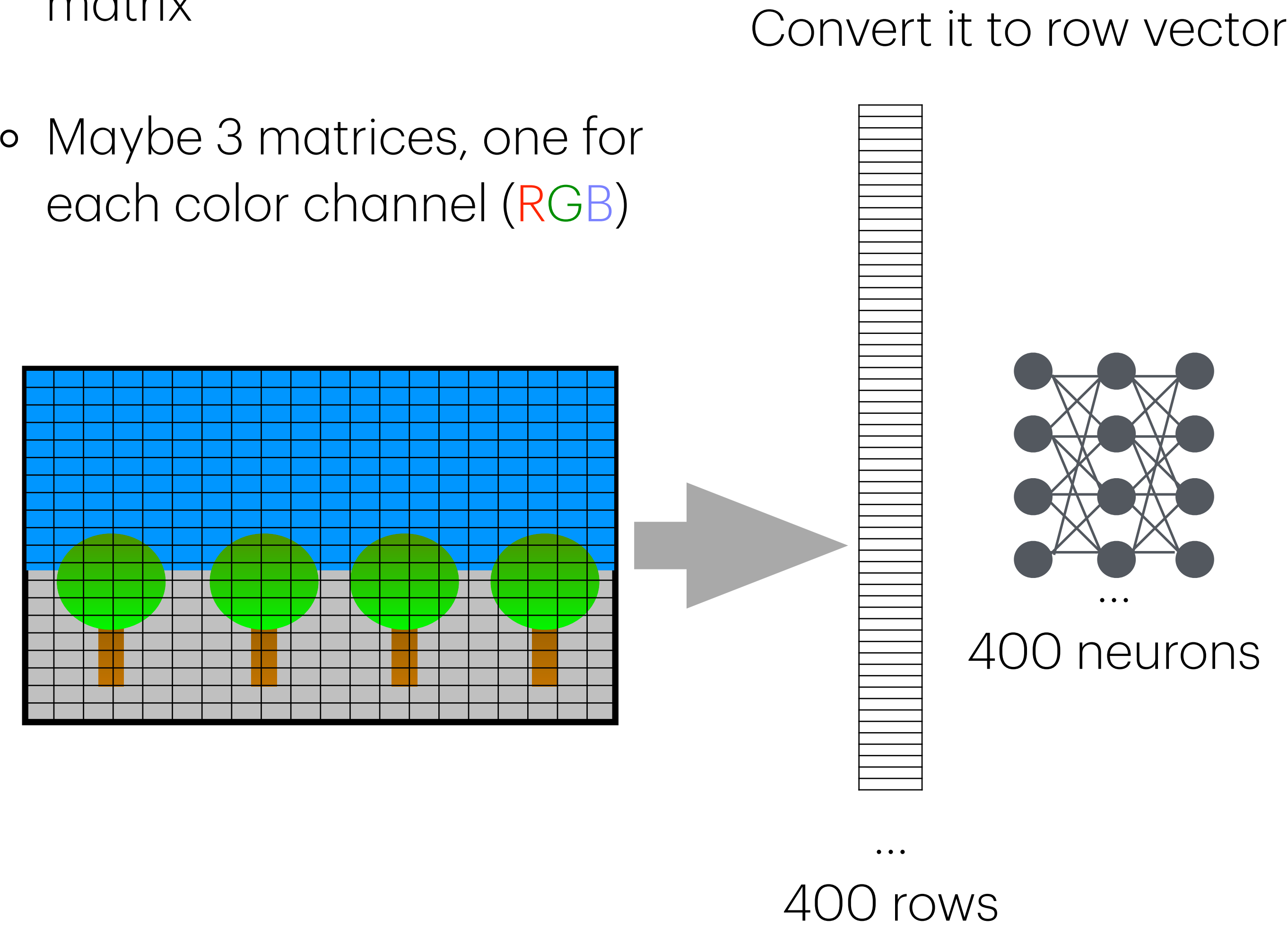
- We see trees
 - We see sky
 - We see ground?
-
- Probably is scene of ?
 - A park?
 - A forest?

Convolutional Neural Netwrks (CNNs): Basics and introduction

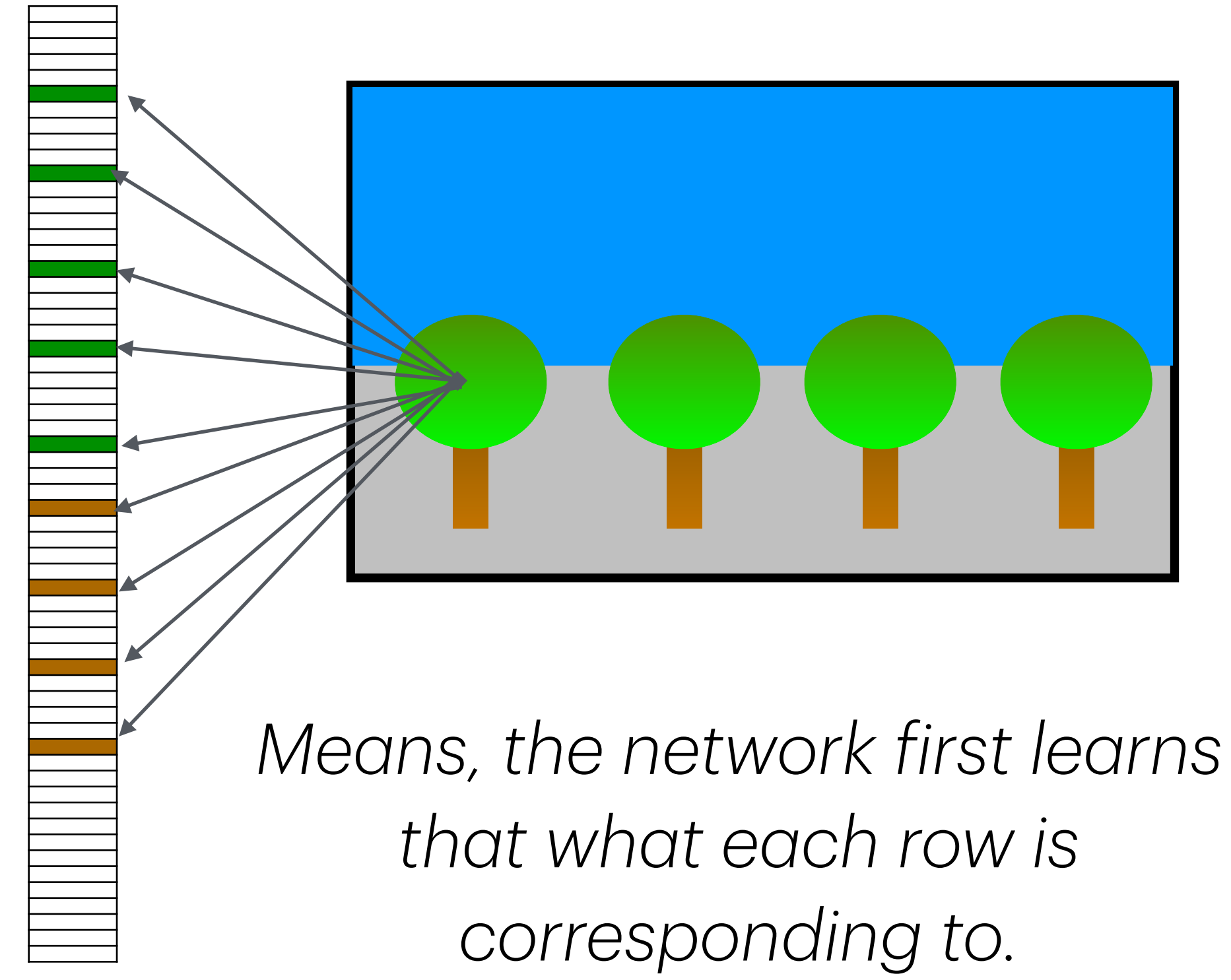
- How **MLPs** would try to understand images?

- e.g. an image is a 20X20 matrix

- Maybe 3 matrices, one for each color channel (RGB)



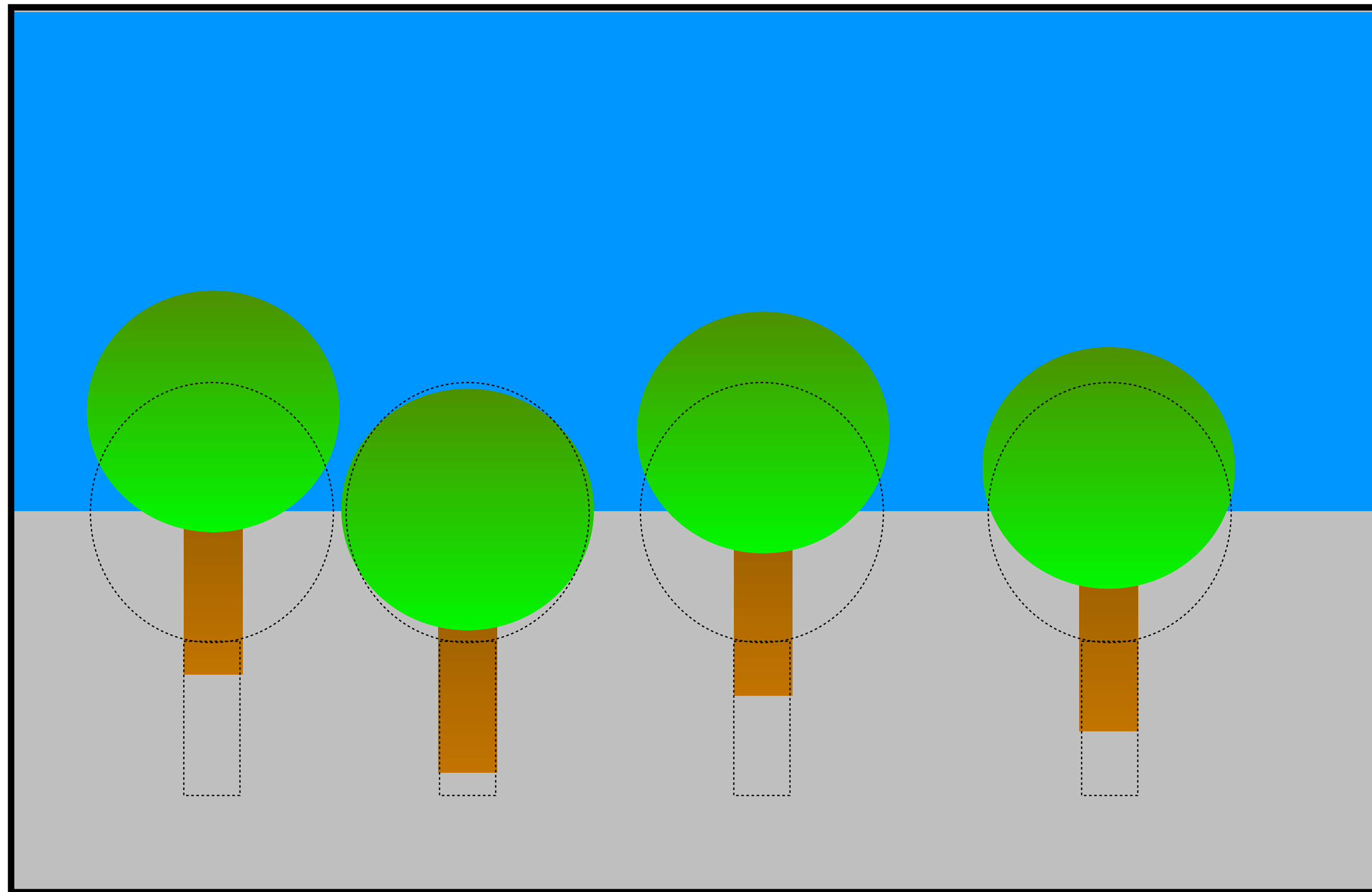
Probably should learn each tree as something like:



Convolutional Neural Netwrks (CNNs): Basics and introduction

- How we (human) understand images?

- **What if the position of trees changes a bit?**



- **Does any of below changes?**

- We see trees
- We see sky
- We see ground?

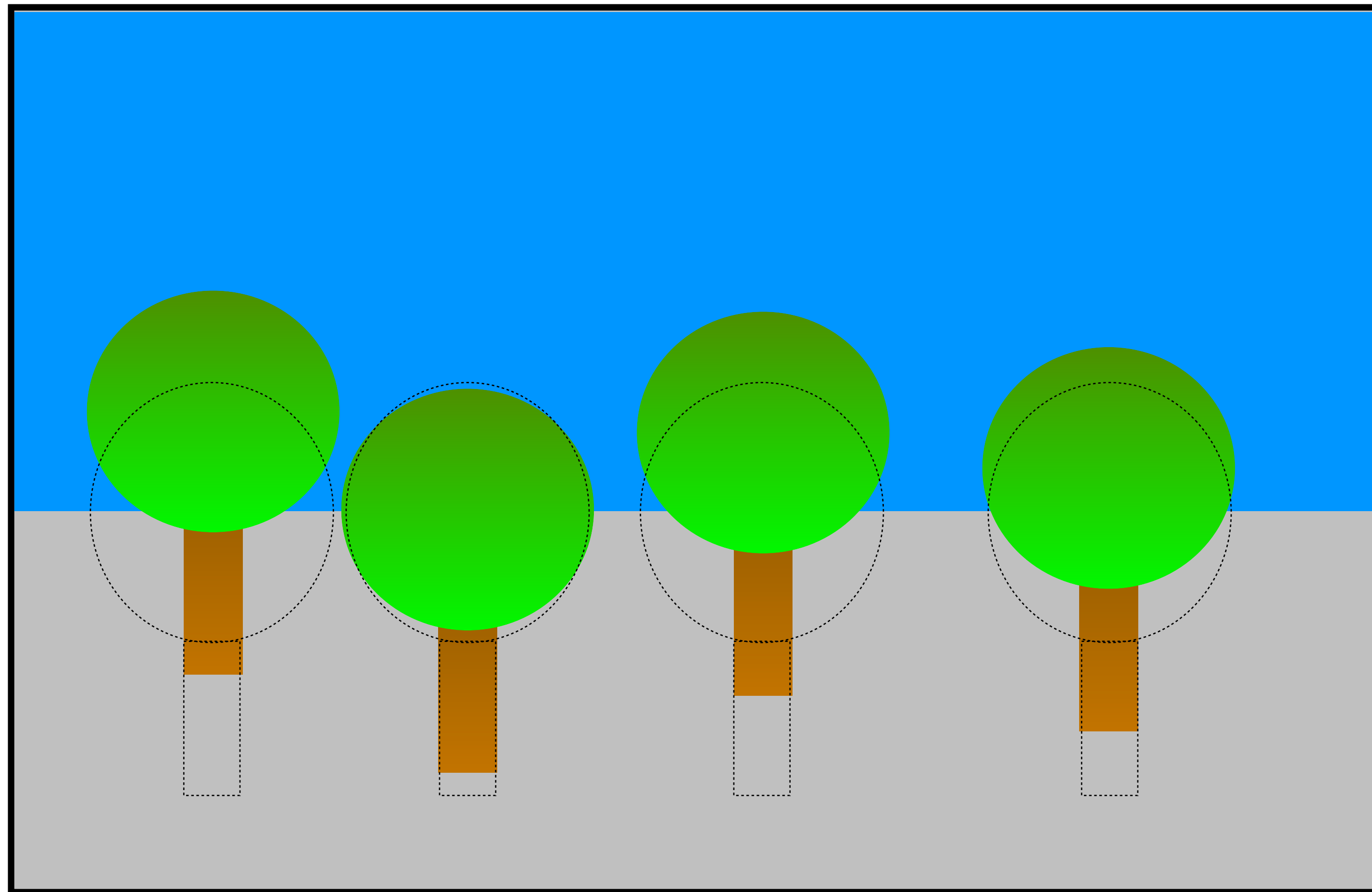
- Probably is scene of ?
- A park?
- A forest?

No!

Convolutional Neural Netwrks (CNNs): Basics and introduction

- How we (human) understand images?

- **What if the position of trees changes a bit?**



- **Does any of below changes?**

- We see trees
- We see sky
- We see ground?

- Probably is scene of ?
- A park?
- A forest?

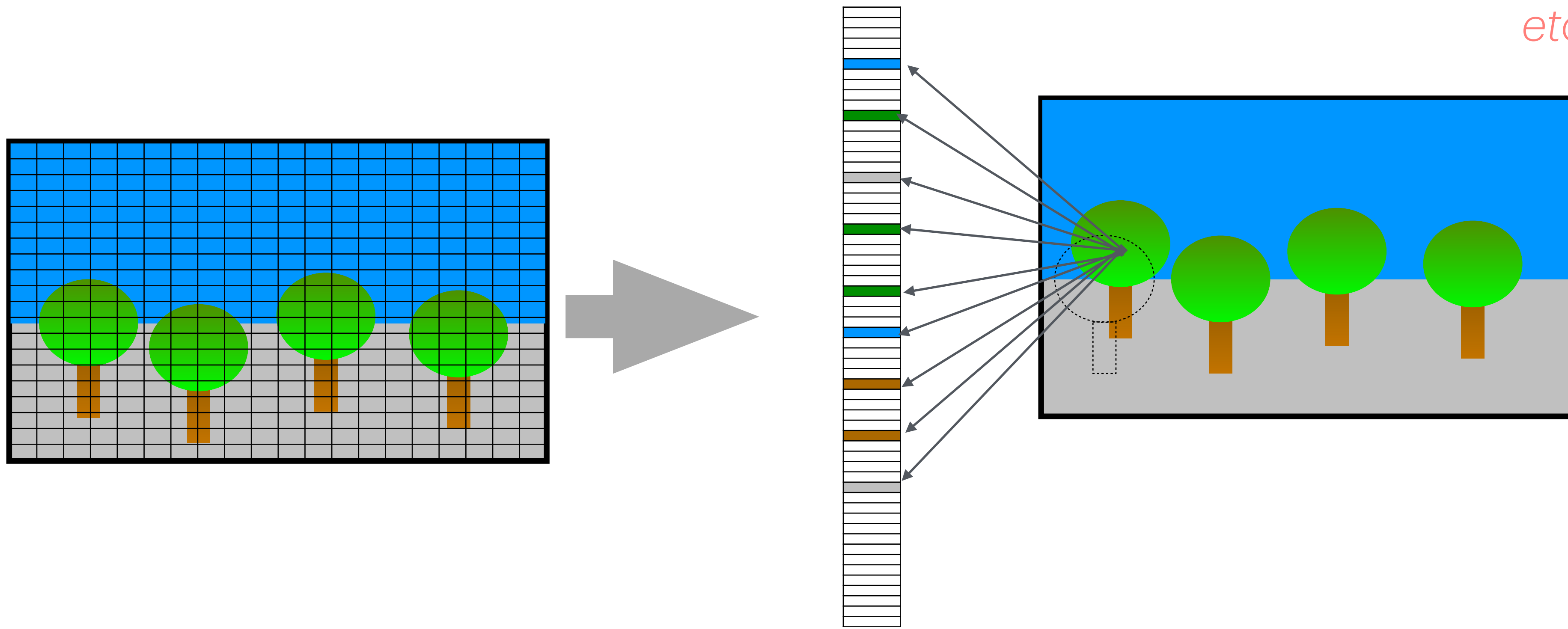
No!

Convolutional Neural Netwrks (CNNs): Basics and introduction

- How **MLPs** would try to understand images?

- **What if the position of trees changes a bit?**

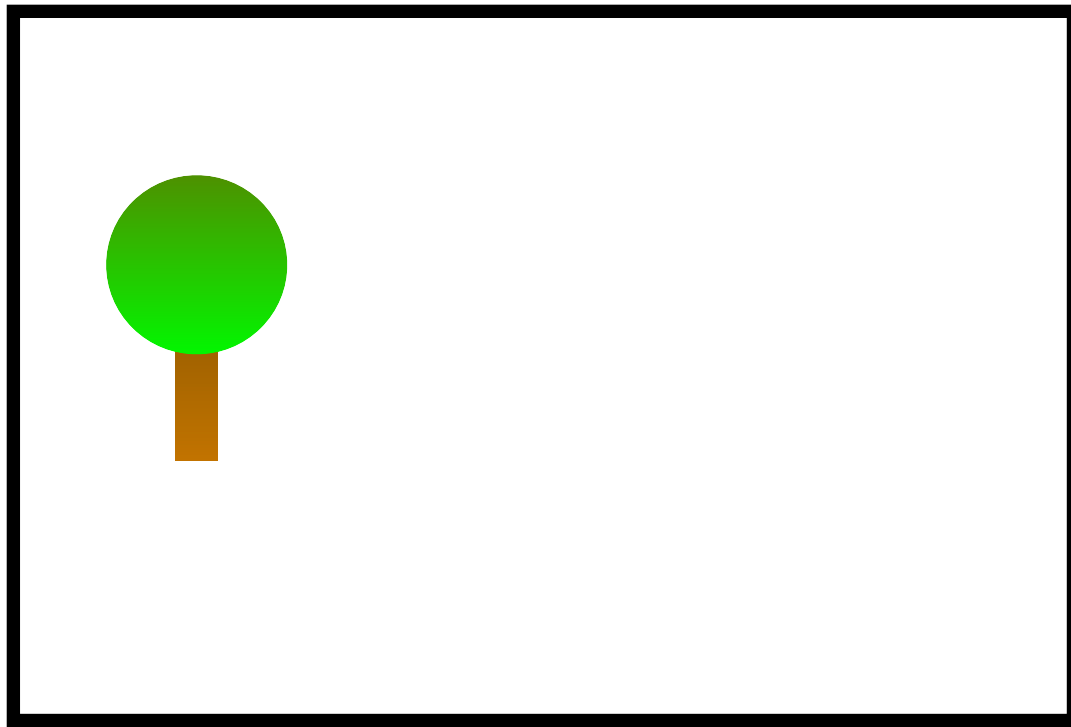
Everything will be messed up, some neurons that would have recognized leaves now are looking at sky or ground etc...



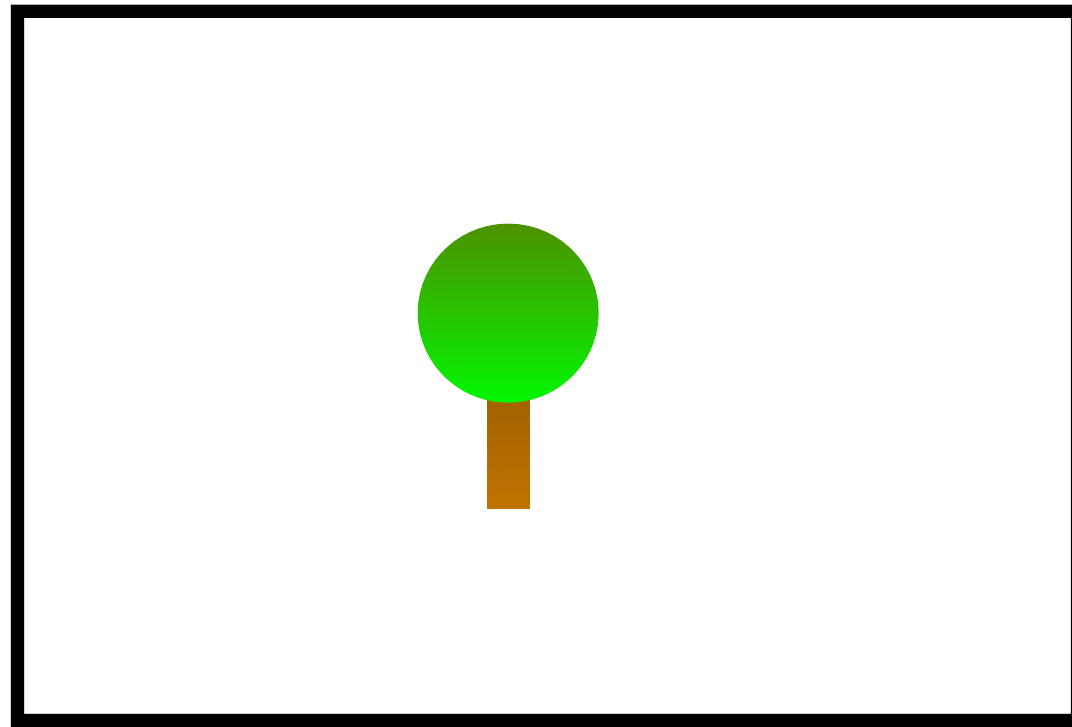
Convolutional Neural Networks (CNNs): Basics and introduction

- Lets analyze what happened, and why MPLs get into trouble
- In an abstract view, images have one deep characteristics: **Translational (shift) symmetry**

This is also a tree



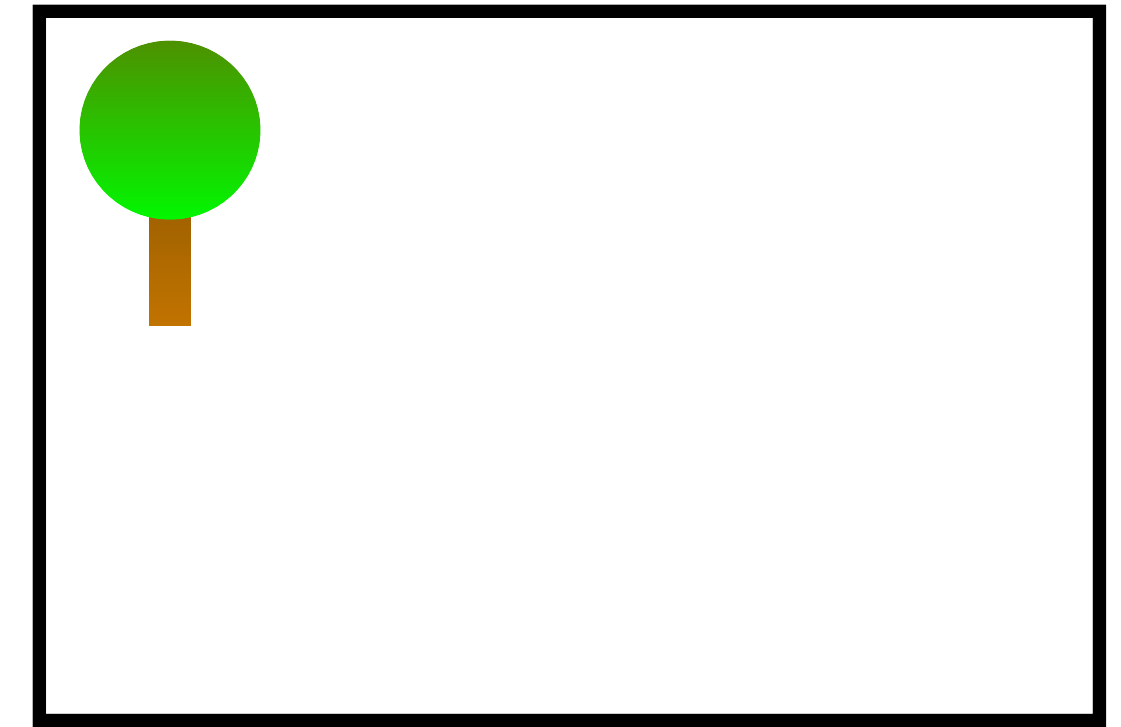
This is also a tree



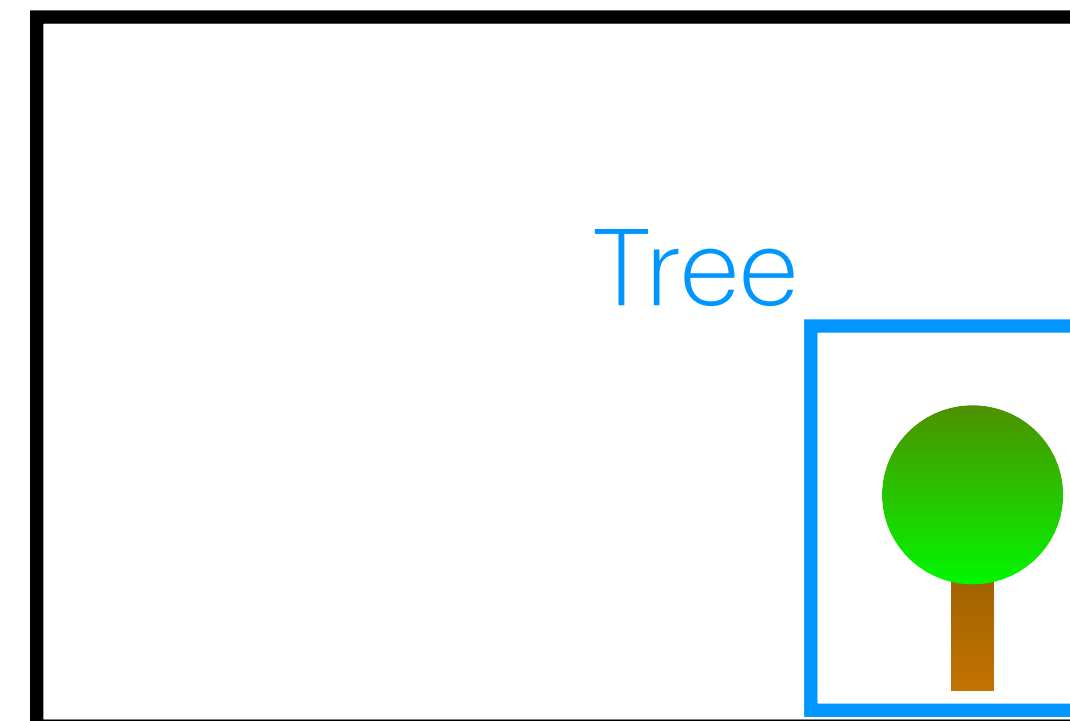
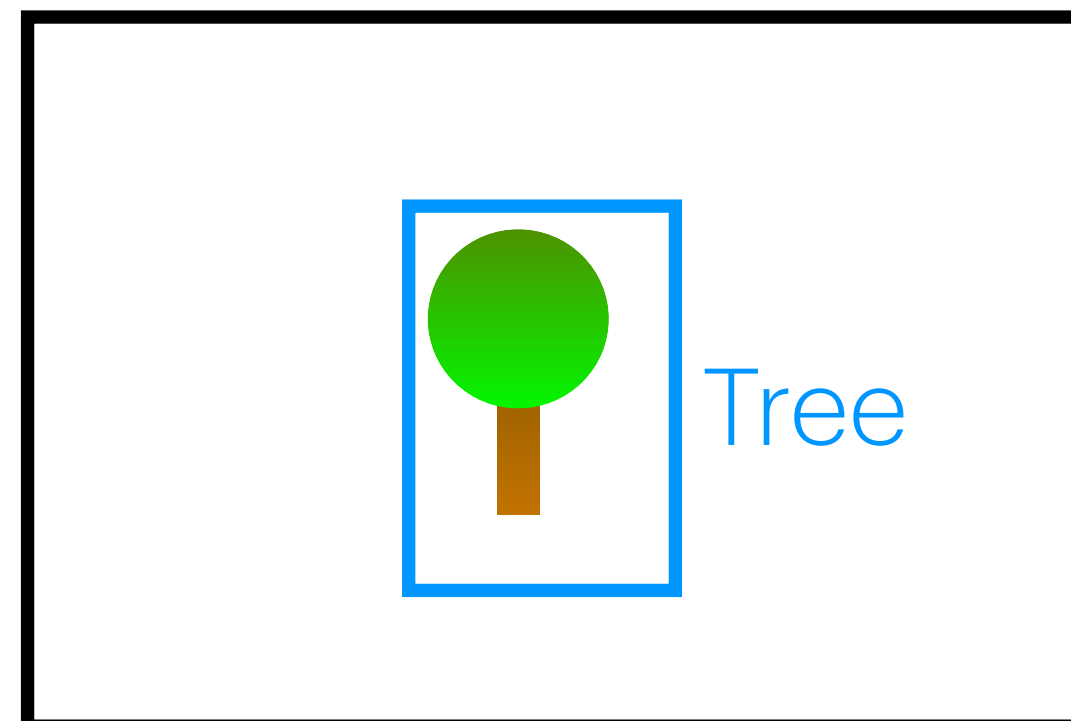
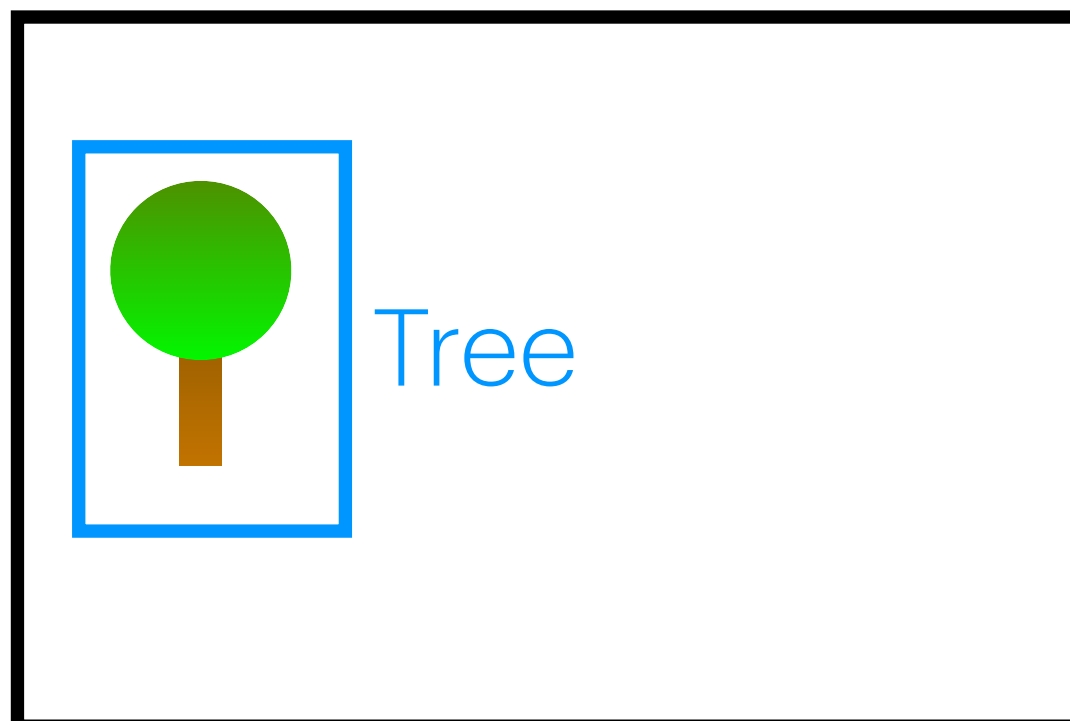
This is also a tree



This is also a tree

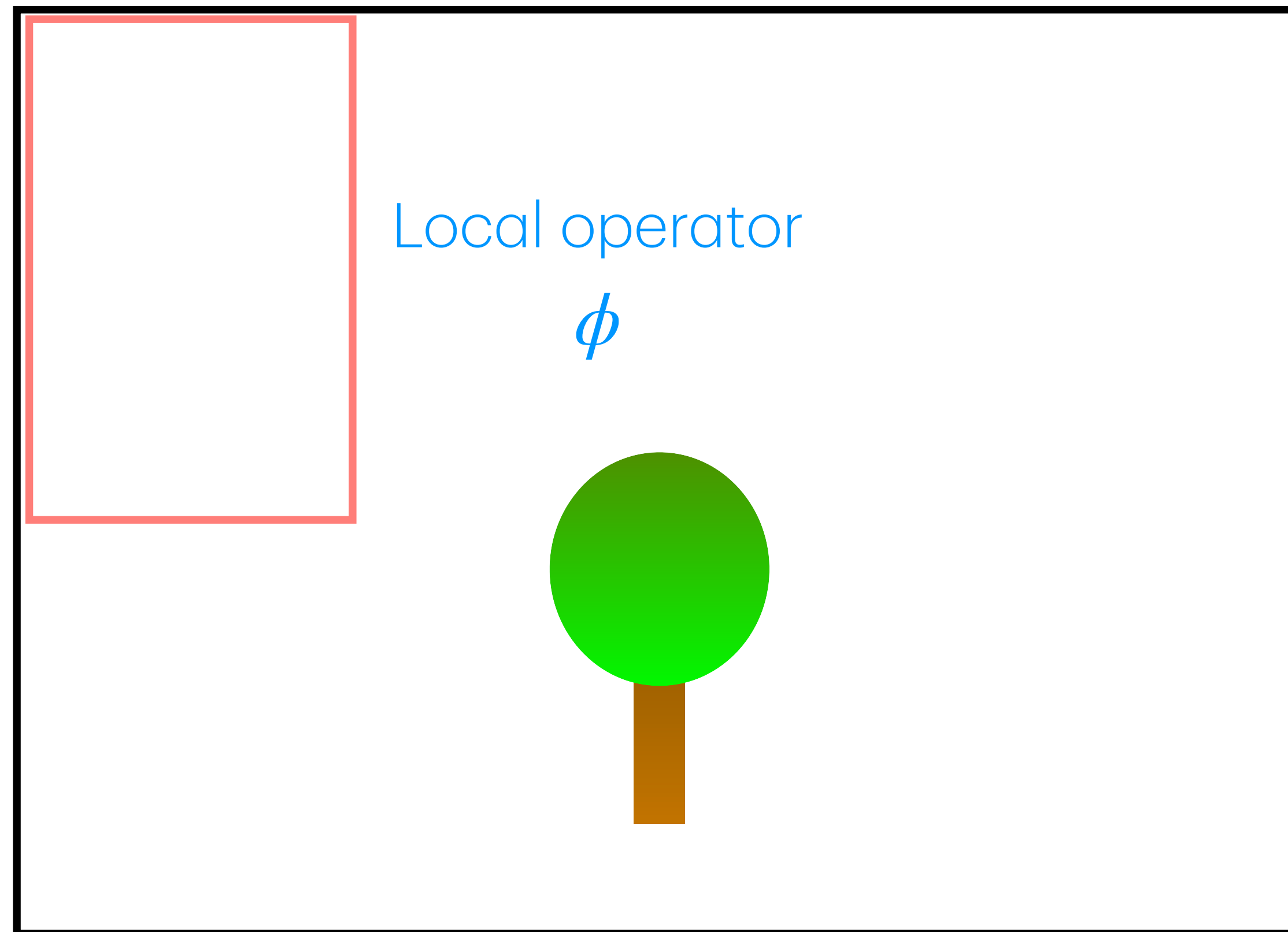


- From the example above, you can see that the best way to identify the tree in the image would be to have a “local” operator that does not care about the positions of the tree!
- Something like:



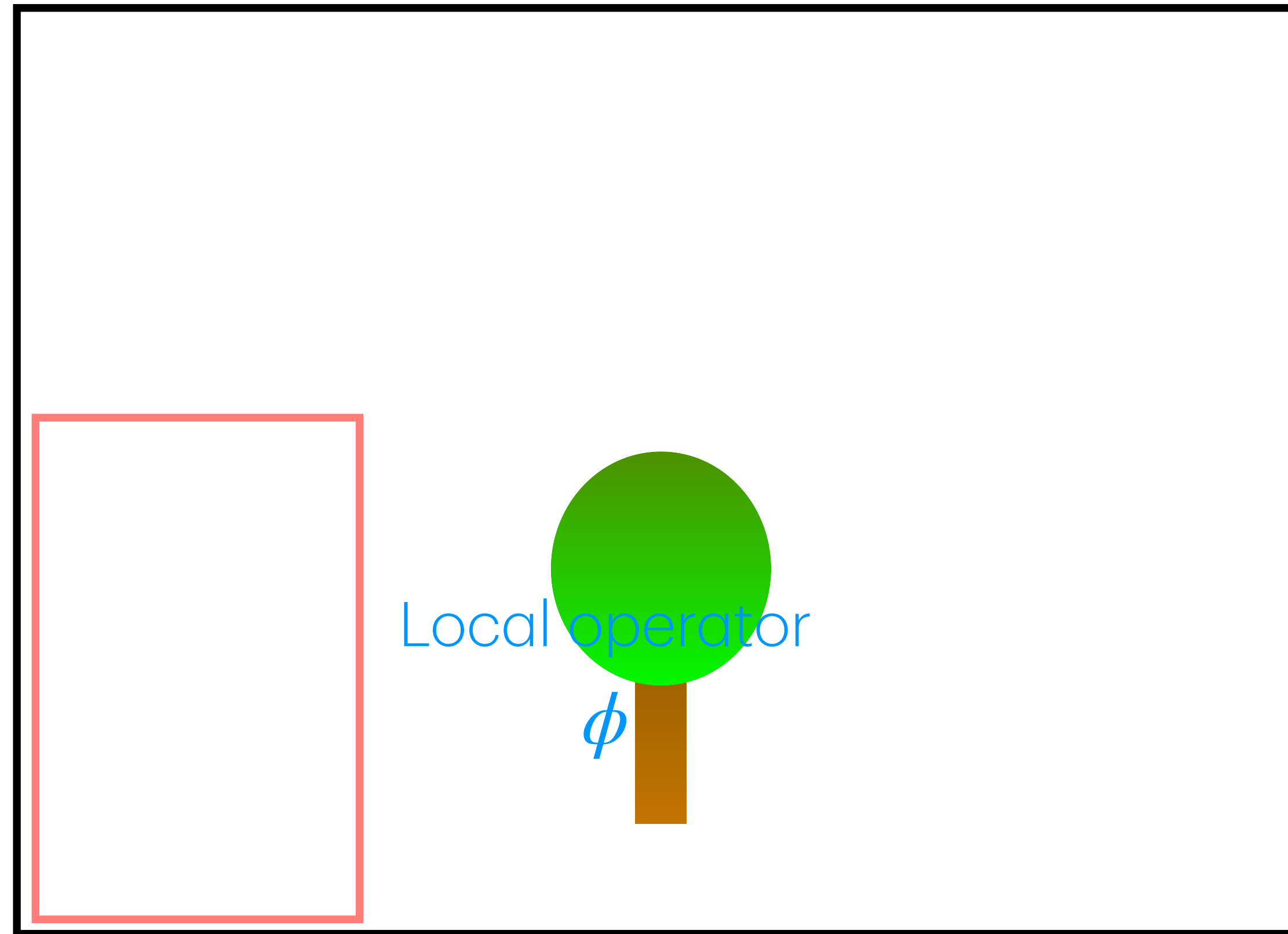
Convolutional Neural Netwrks (CNNs): Basics and introduction

- So what we will do next, is to define such a local operator, that can identify objects locally, and then will slide it around to find objects, or more generally local features in our input:



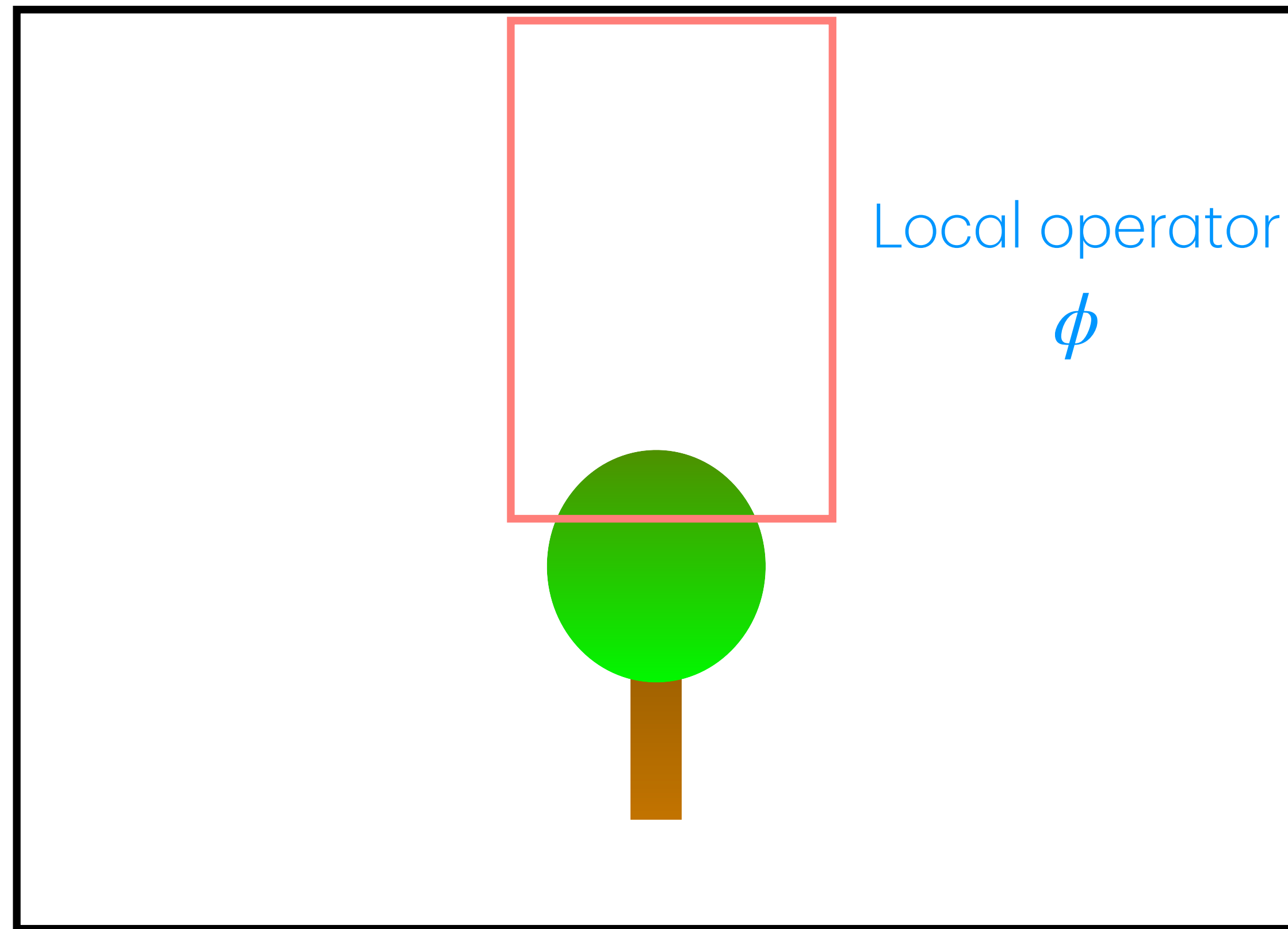
Convolutional Neural Netwrks (CNNs): Basics and introduction

- So what we will do next, is to define such a local operator, that can identify objects locally, and then will slide it around to find objects, or more generally local features in our input:



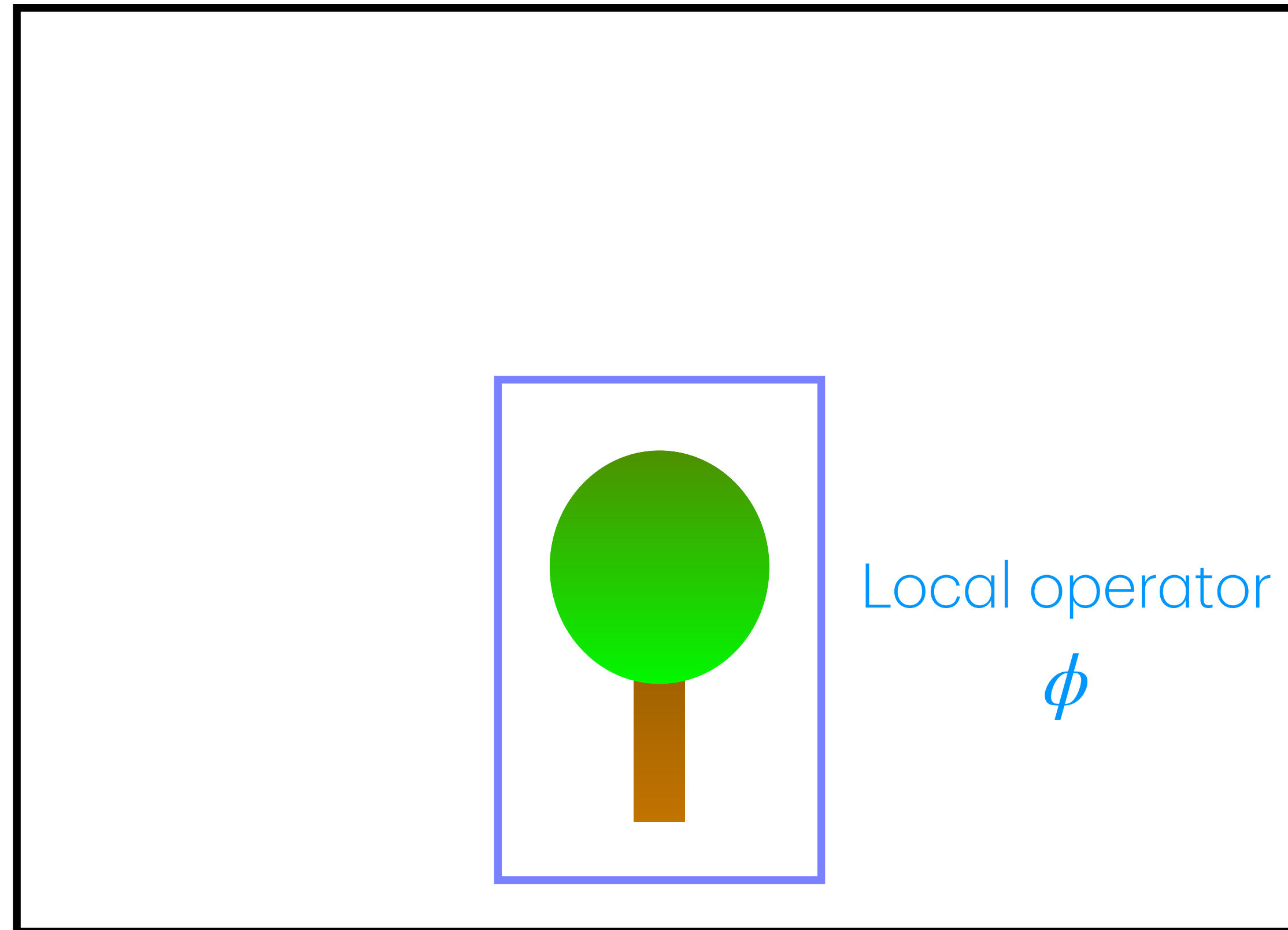
Convolutional Neural Networks (CNNs): Basics and introduction

- So what we will do next, is to define such a local operator, that can identify objects locally, and then will slide it around to find objects, or more generally local features in our input:



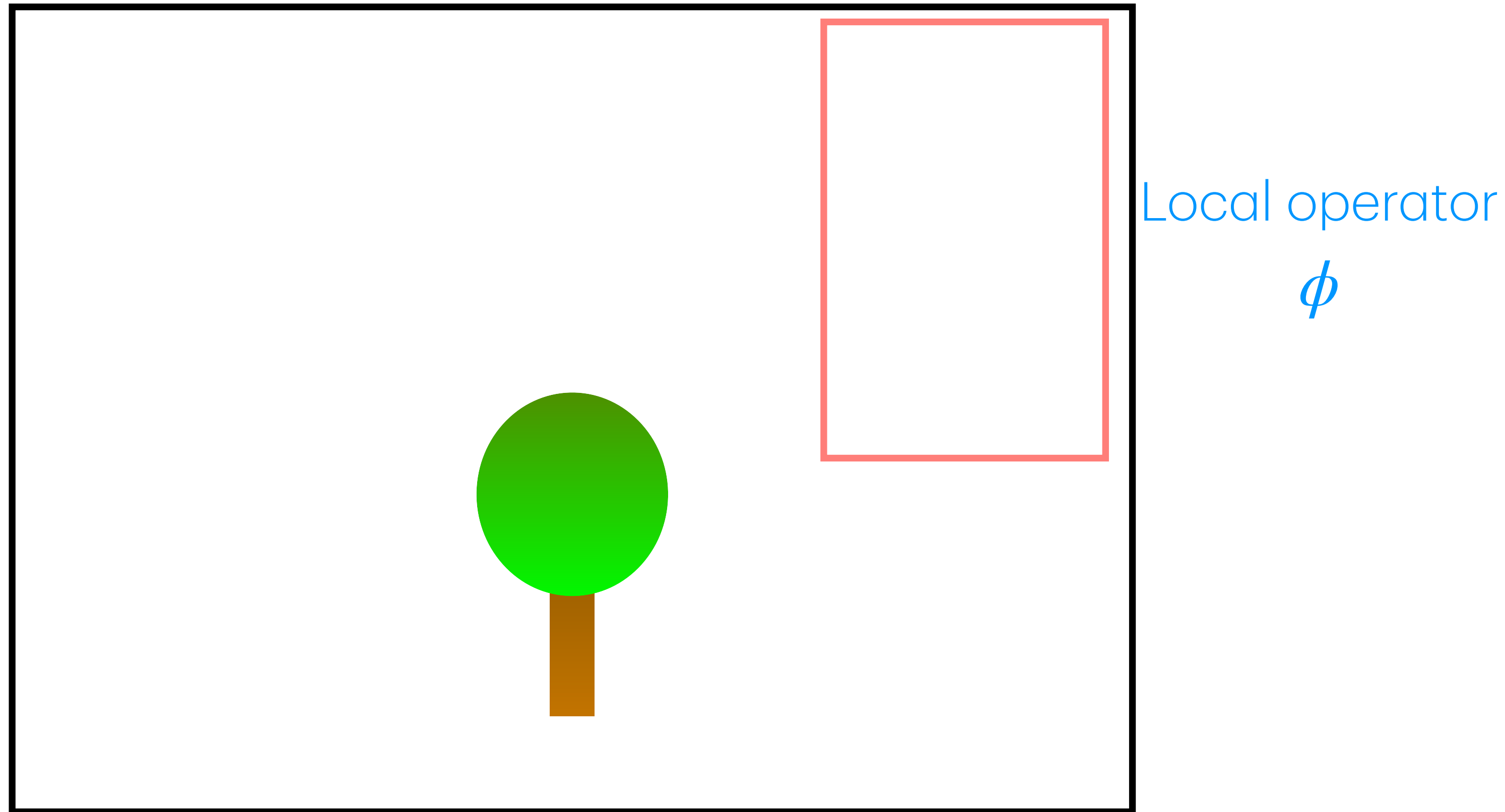
Convolutional Neural Networks (CNNs): Basics and introduction

- So what we will do next, is to define such a local operator, that can identify objects locally, and then will slide it around to find objects, or more generally local features in our input:



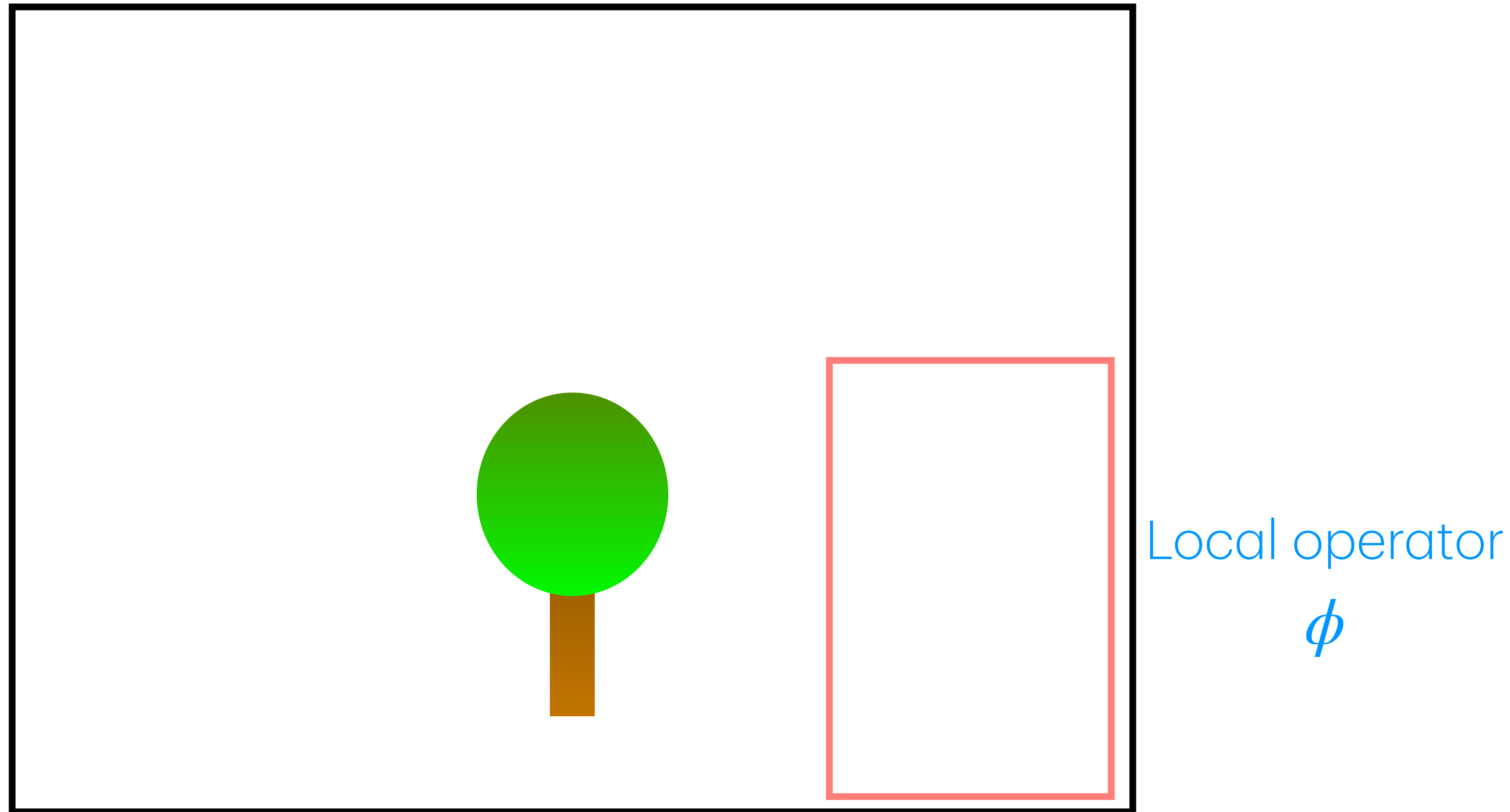
Convolutional Neural Networks (CNNs): Basics and introduction

- So what we will do next, is to define such a local operator, that can identify objects locally, and then will slide it around to find objects, or more generally local features in our input:



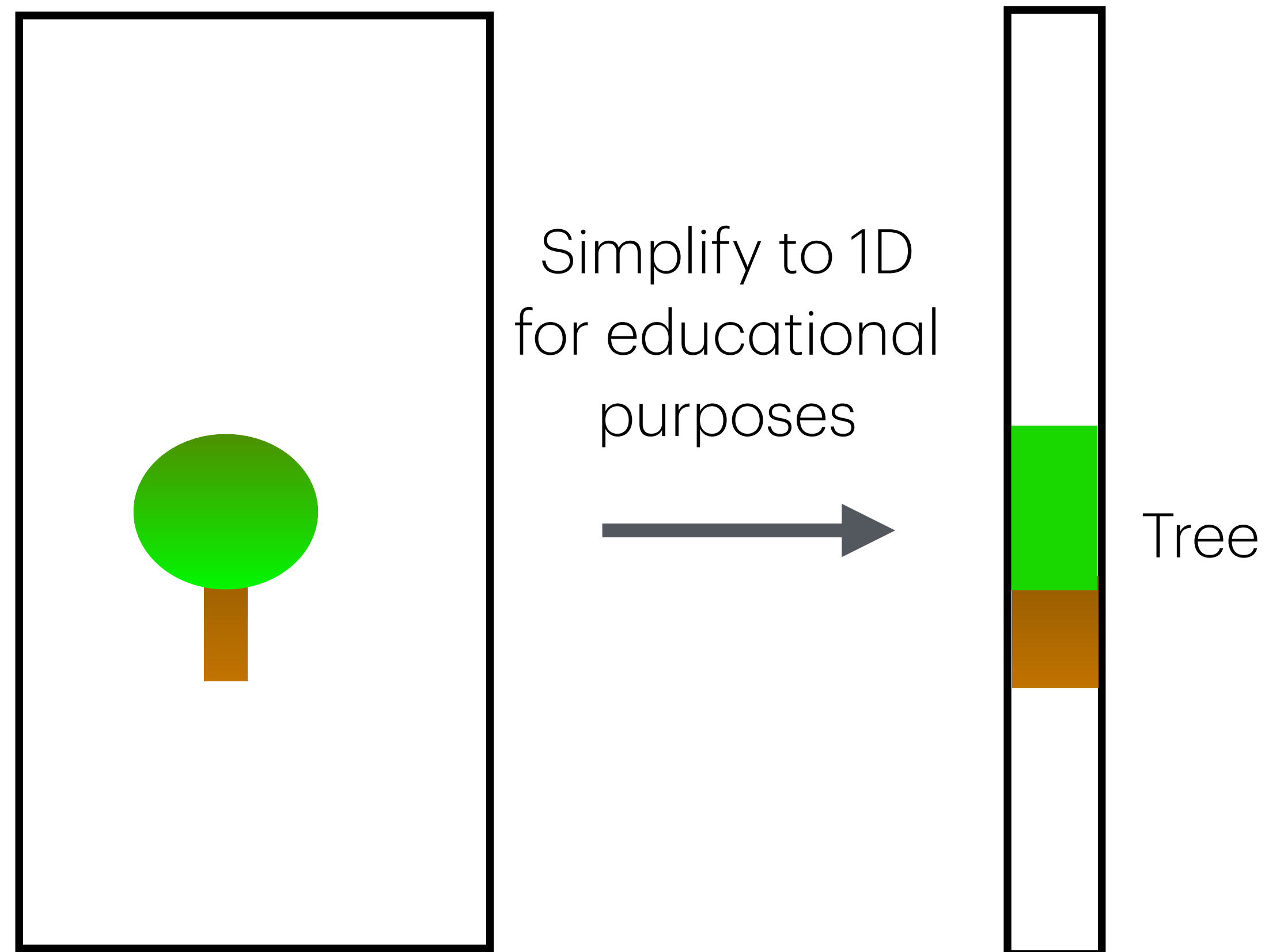
Convolutional Neural Networks (CNNs): Basics and introduction

- So what we will do next, is to define such a local operator, that can identify objects locally, and then will slide it around to find objects, or more generally local features in our input:



Convolutional Neural Networks (CNNs): Basics and introduction

- The same scenario still applies, even if our input was not a 2D matrix (an image).
- To make notation and visualization more intuitive, let's consider 1D matrix as our input.



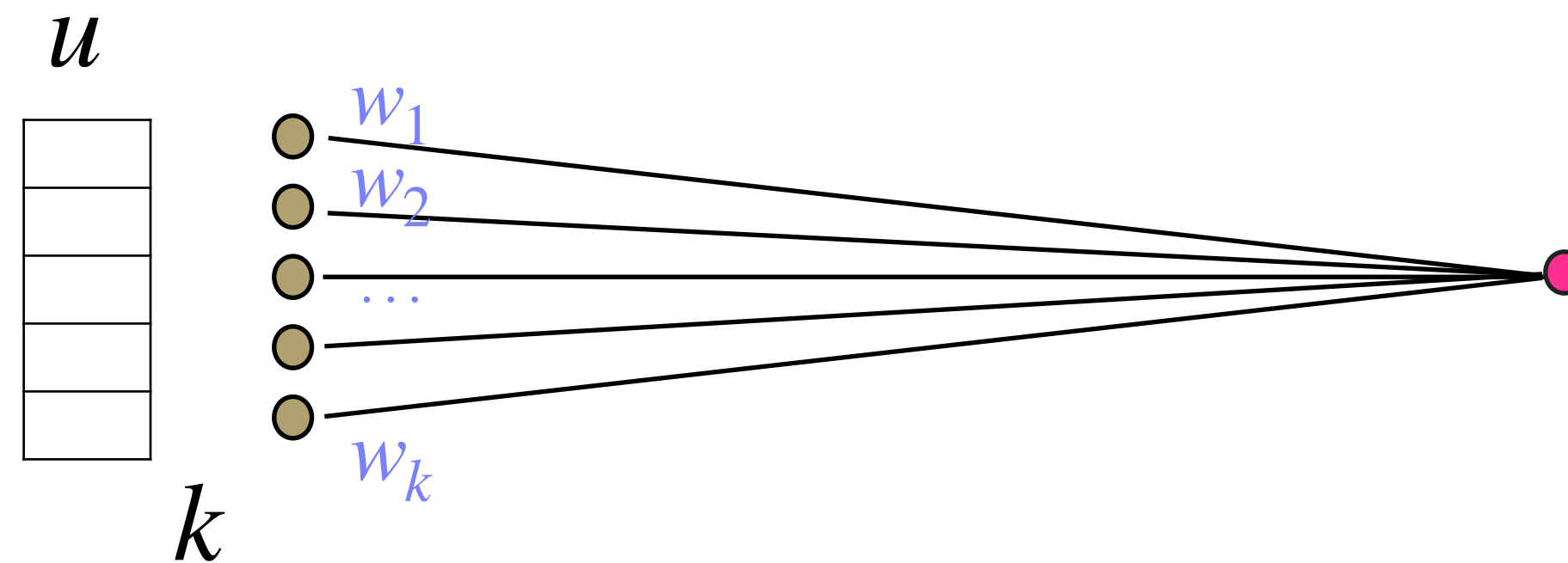
- Now let's look at the same problem again:

- In our 1D world, where our image has the dimension N , we can think of a **local linear operator**, one that only looks at a small window u of size k of the input, where $k \ll N$.
- Mathematically, ϕ computes a dot product between the input window $u \in \mathbb{R}^k$ and a learned weight vector $w \in \mathbb{R}^k$:

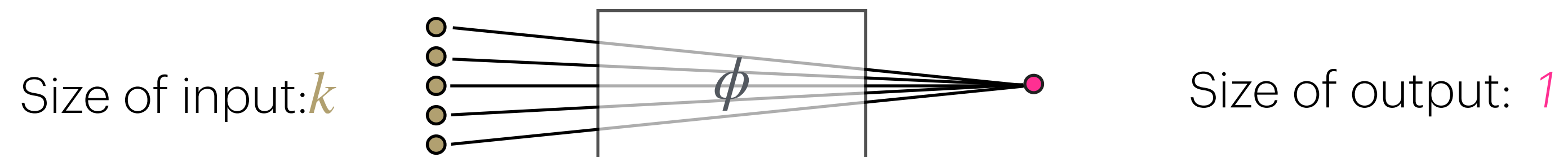
$$\phi : \mathbb{R}^k \rightarrow \mathbb{R}$$

$$\phi(u) = u \cdot w = u_1 w_1 + u_2 w_2 + \dots + u_k w_k$$

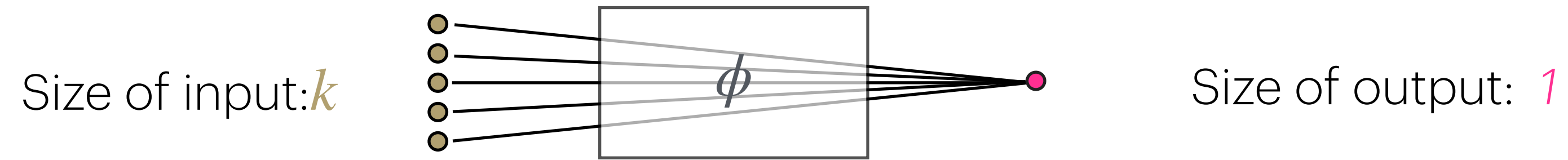
- Computation graph:



- Visually we show it like:



Concept 1: Linear Convolutional Operator



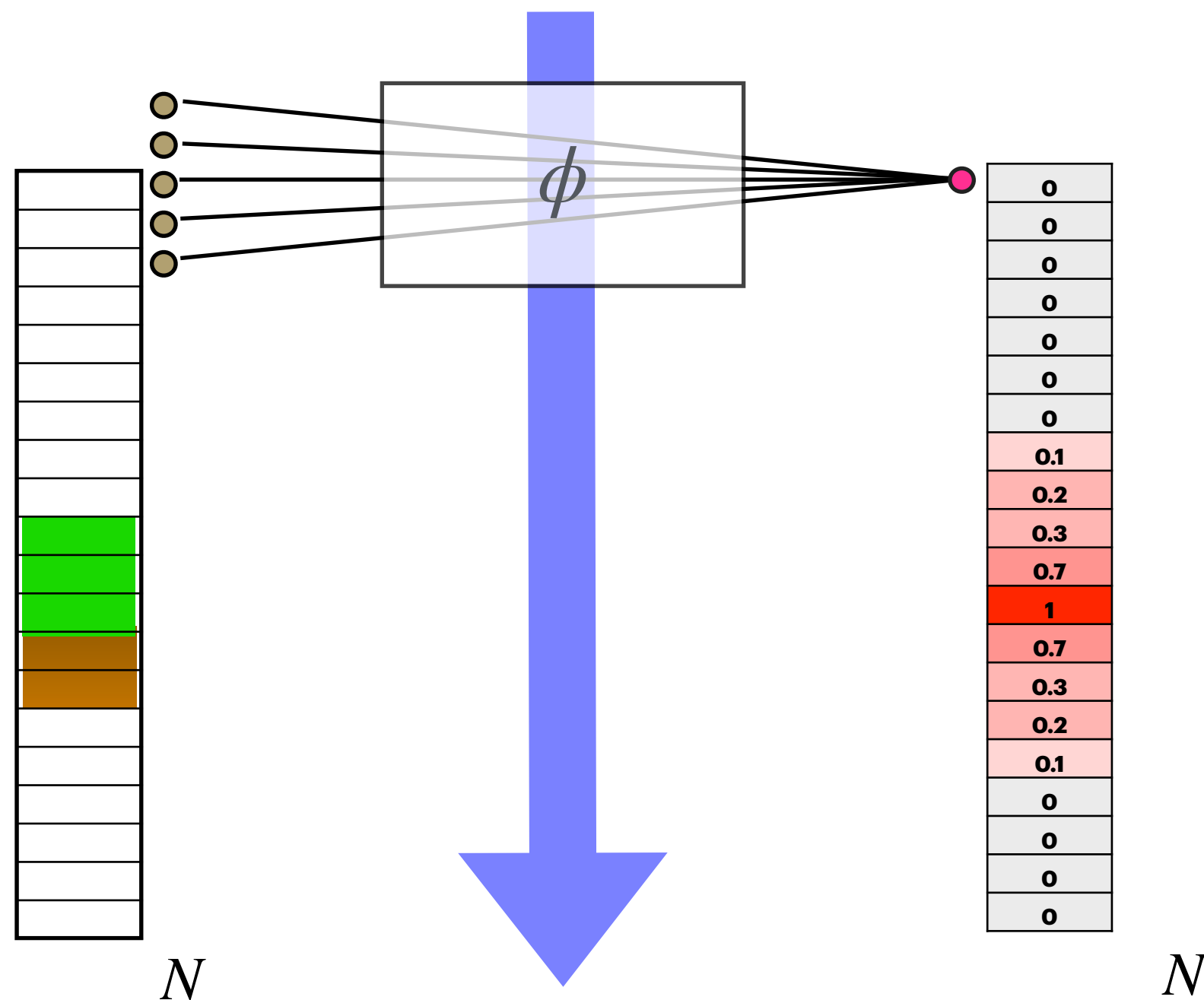
$$\phi : \mathbb{R}^k \rightarrow \mathbb{R}$$

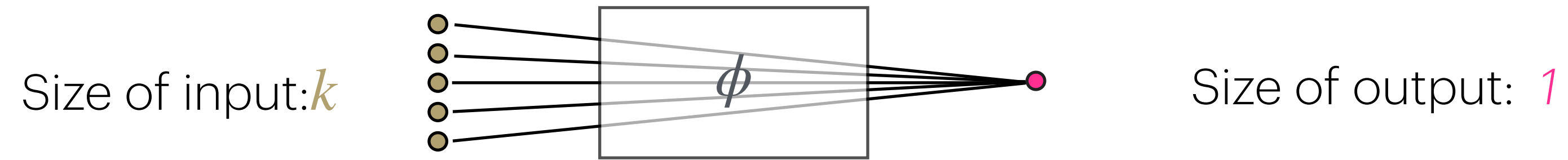
$$\phi(u) = u \cdot w = u_1 w_1 + u_2 w_2 + \dots + u_k w_k$$

- If we **slide** the local operator ϕ over our input, something like this would happen:

Is the input window a tree?

Sliding





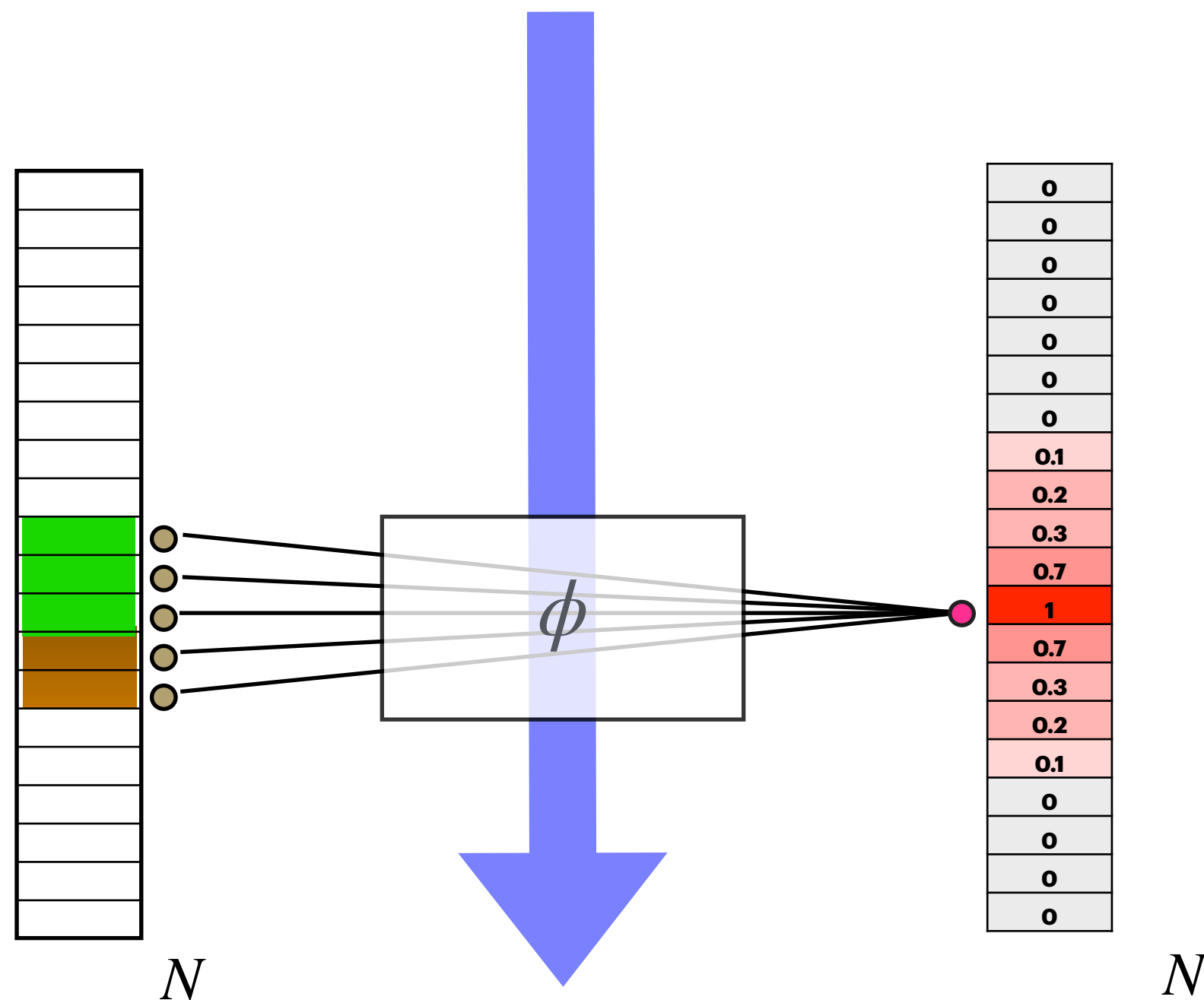
$$\phi : \mathbb{R}^k \rightarrow \mathbb{R}$$

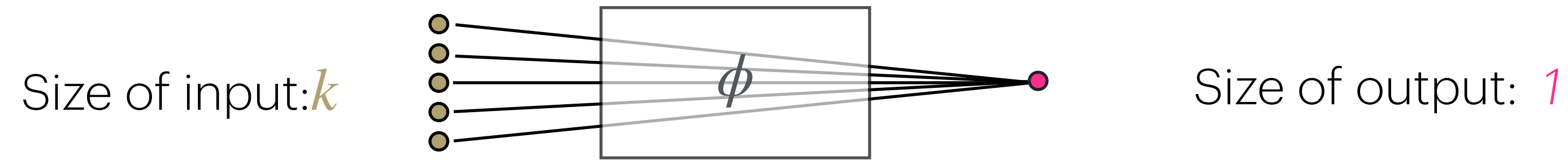
$$\phi(u) = u \cdot w = u_1 w_1 + u_2 w_2 + \dots + u_k w_k$$

- If we **slide** the local operator ϕ over our input, something like this would happen:

Is the input window a tree?

Sliding





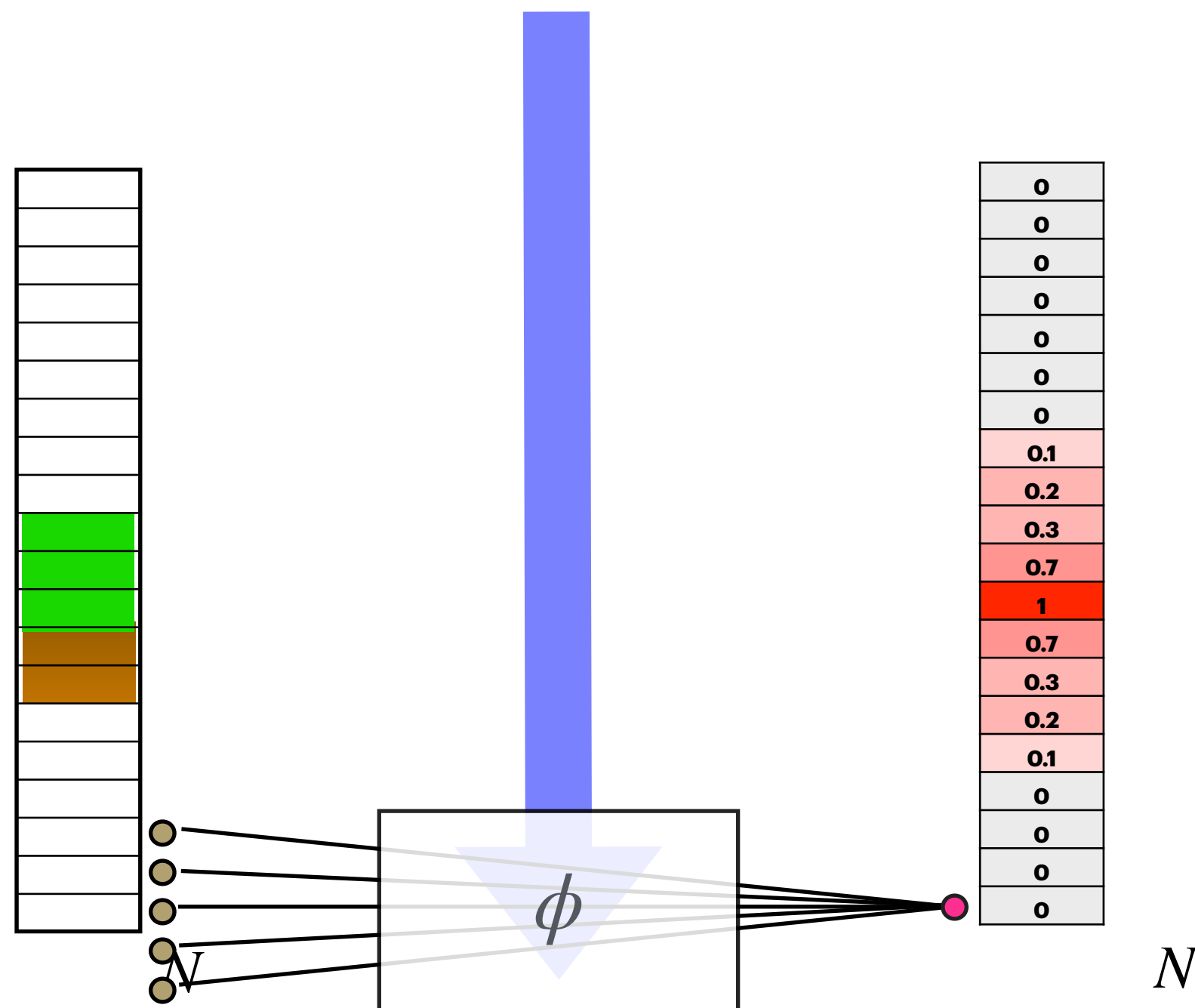
$$\phi : \mathbb{R}^k \rightarrow \mathbb{R}$$

$$\phi(u) = u \cdot w = u_1 w_1 + u_2 w_2 + \dots + u_k w_k$$

- If we **slide** the local operator ϕ over our input, something like this would happen:

Is the input window a tree?

Sliding

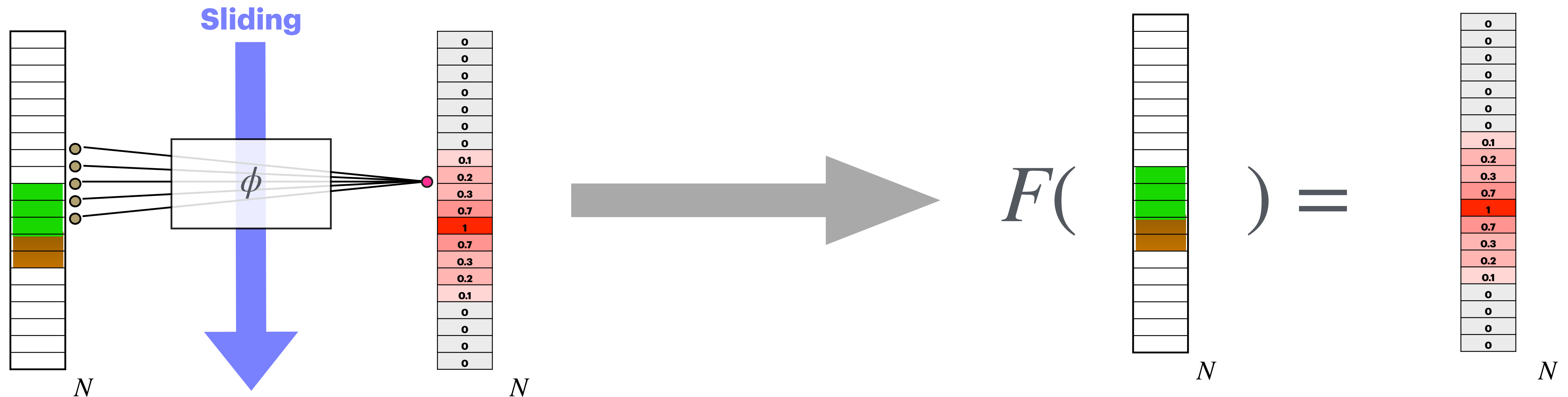


- Note that the linear weights w , are learnable via backpropagation**

- We can combine these two operators into a function, and call it **F**:

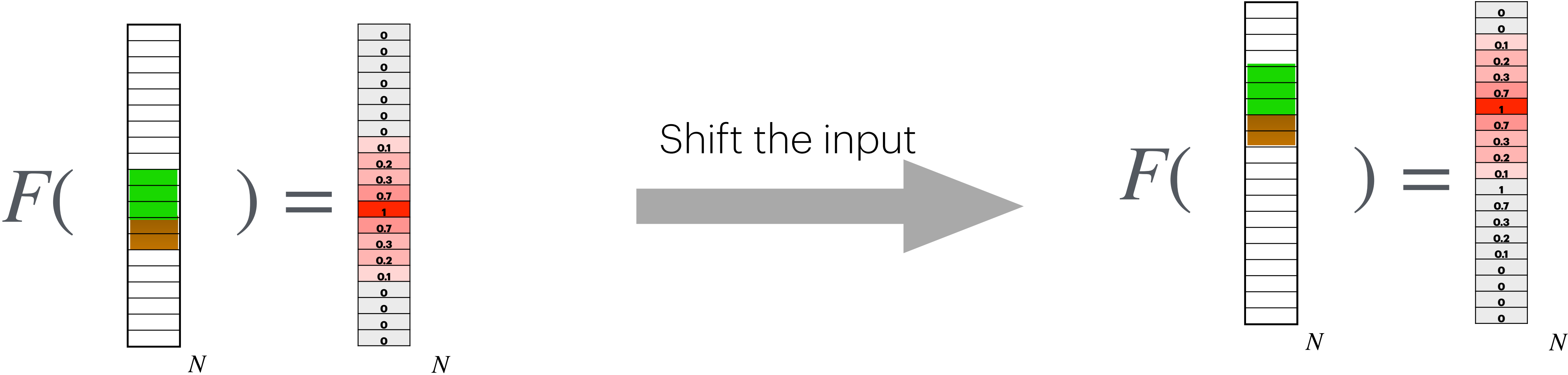
F = Sliding the local operator (known as *kernel*) ϕ on the whole input

$$F : \mathbb{R}^N \rightarrow \mathbb{R}^{N'}$$



- What does Function **F** do?
 - It probabilistically locates the position of the tree in our input, and points it out!

- Function **F** also has a very unique feature:
 - If we apply a shift (translation) to the input of **F**, the output of **F** changes exactly according to the shift.



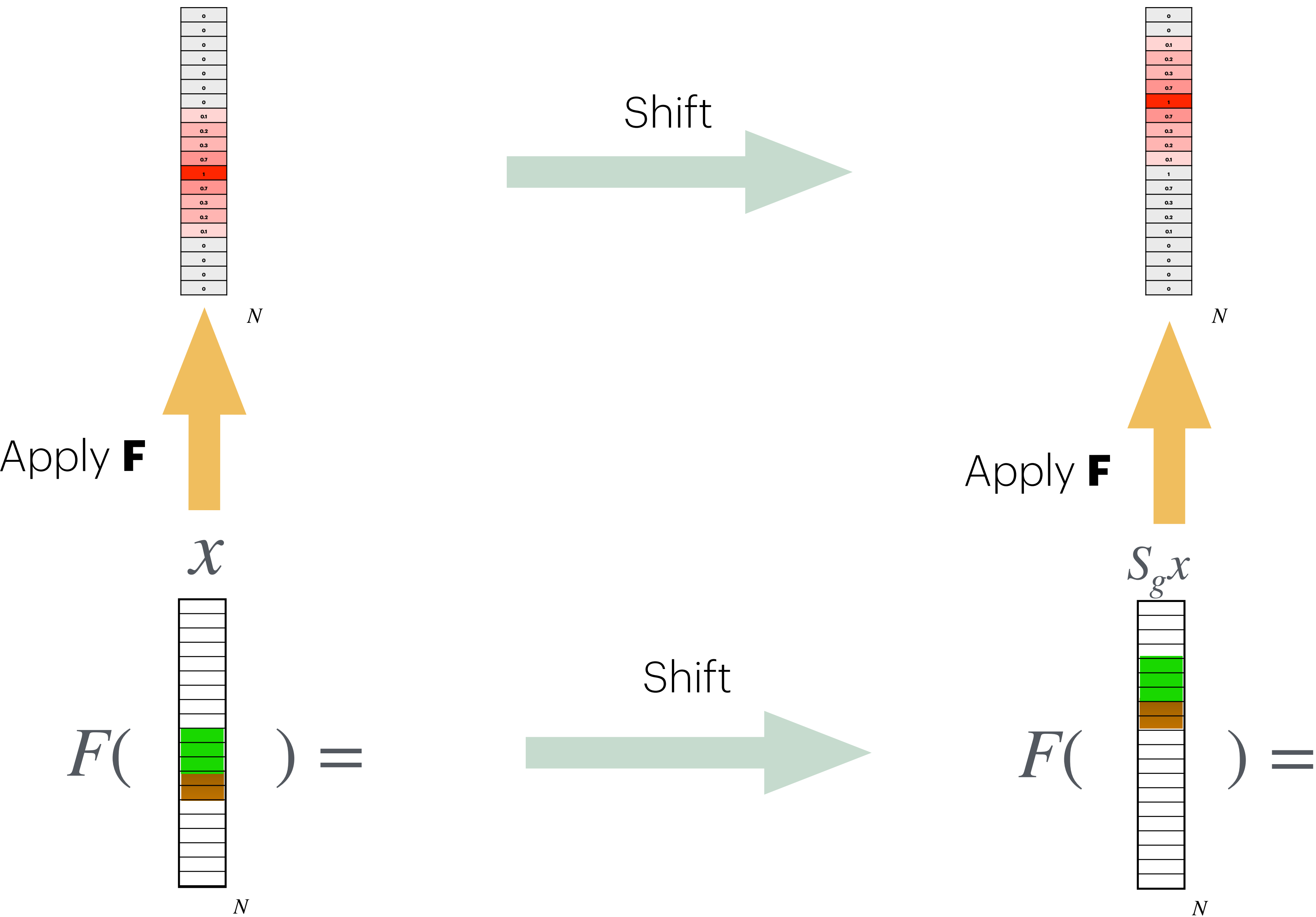
- Mathematically: The Shift operator that shifts the input by g , is defined as:

$$S_g \text{ for } g \in \mathbb{Z}$$

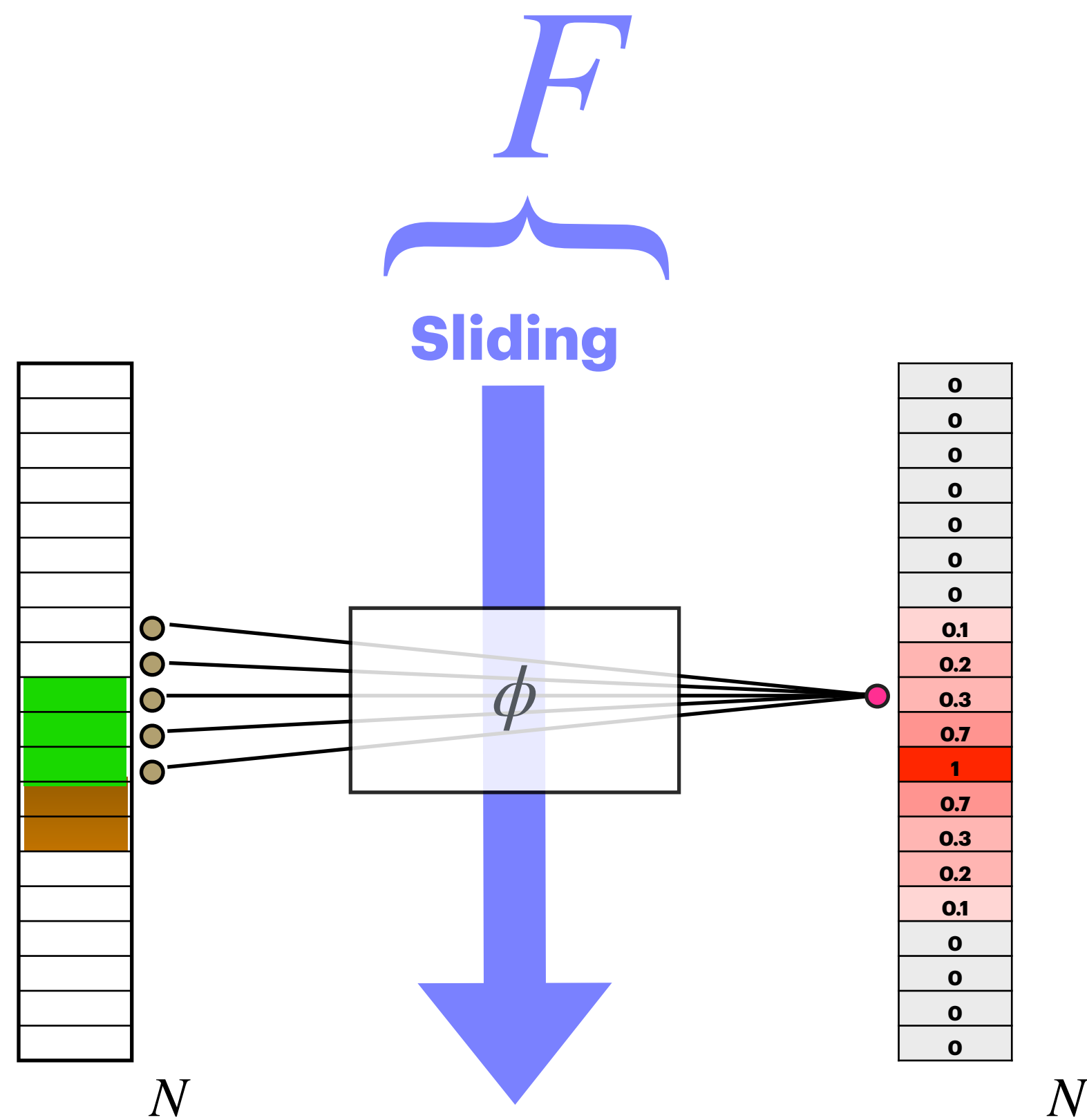
Then the function **F** has this unique feature: **F** is **shift Equivariance**

$$F(S_g x) = S_g F(x)$$

- Visually if F is Equivariance ($F(S_g x) = S_g F(x)$) means:



- Let's go back to our motivation:
Instead of feeding the whole input into an large MLP, we defined a smarter operator: a local kernel ϕ that slides over the entire input. We called this operator **F**.



- In standard ML notation, F is called a convolutional operator, and the operation is written as:

$$F(x) = \phi \star x \quad , \quad F : \mathbb{R}^N \rightarrow \mathbb{R}^{N'}$$

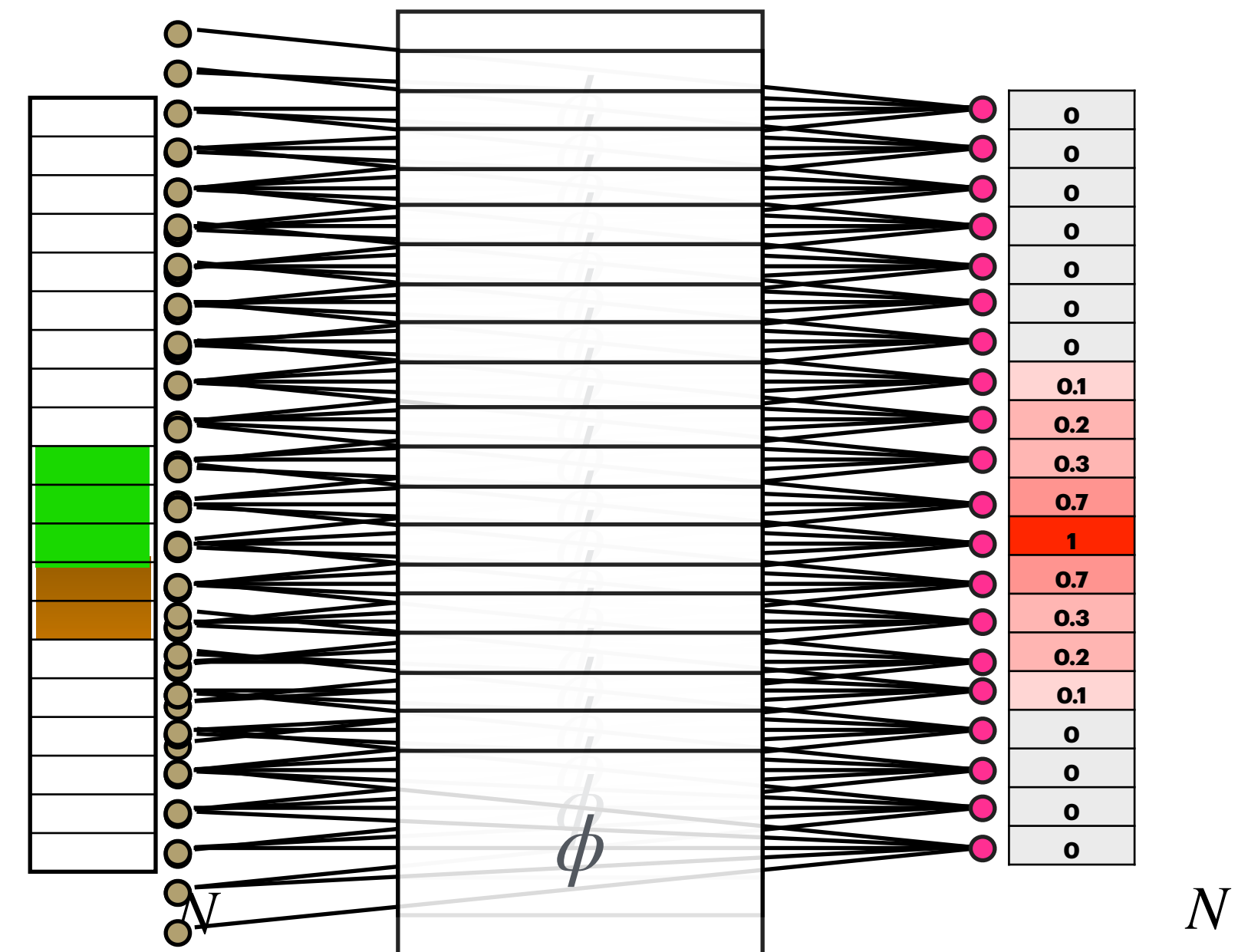
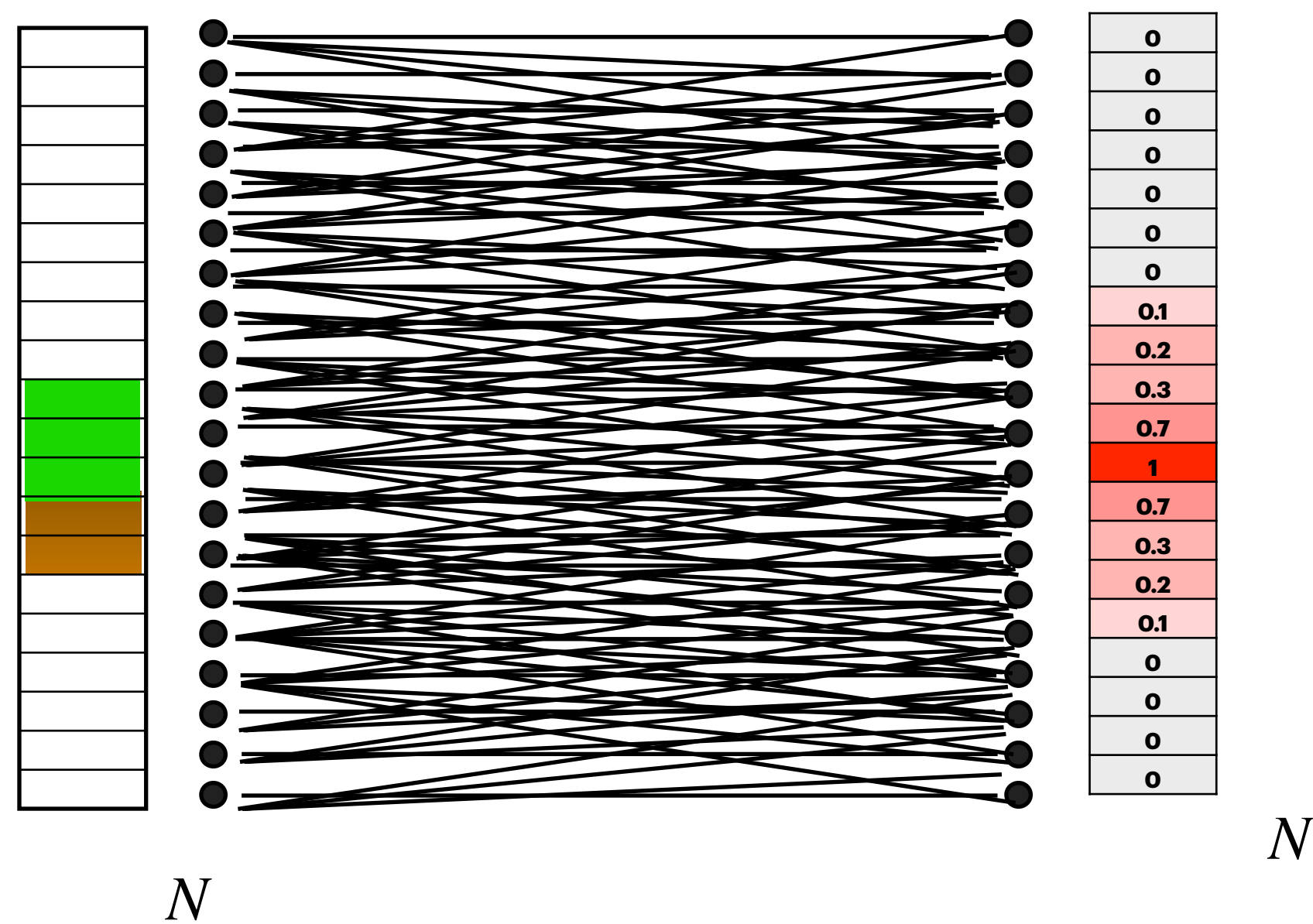
This notation means exactly what we described: slide the kernel ϕ (which is a linear function) over every position of x . That's it.

And F is a translation-equivariant operator:

$$F(S_g x) = S_g F(x) .$$

|| Pause Point

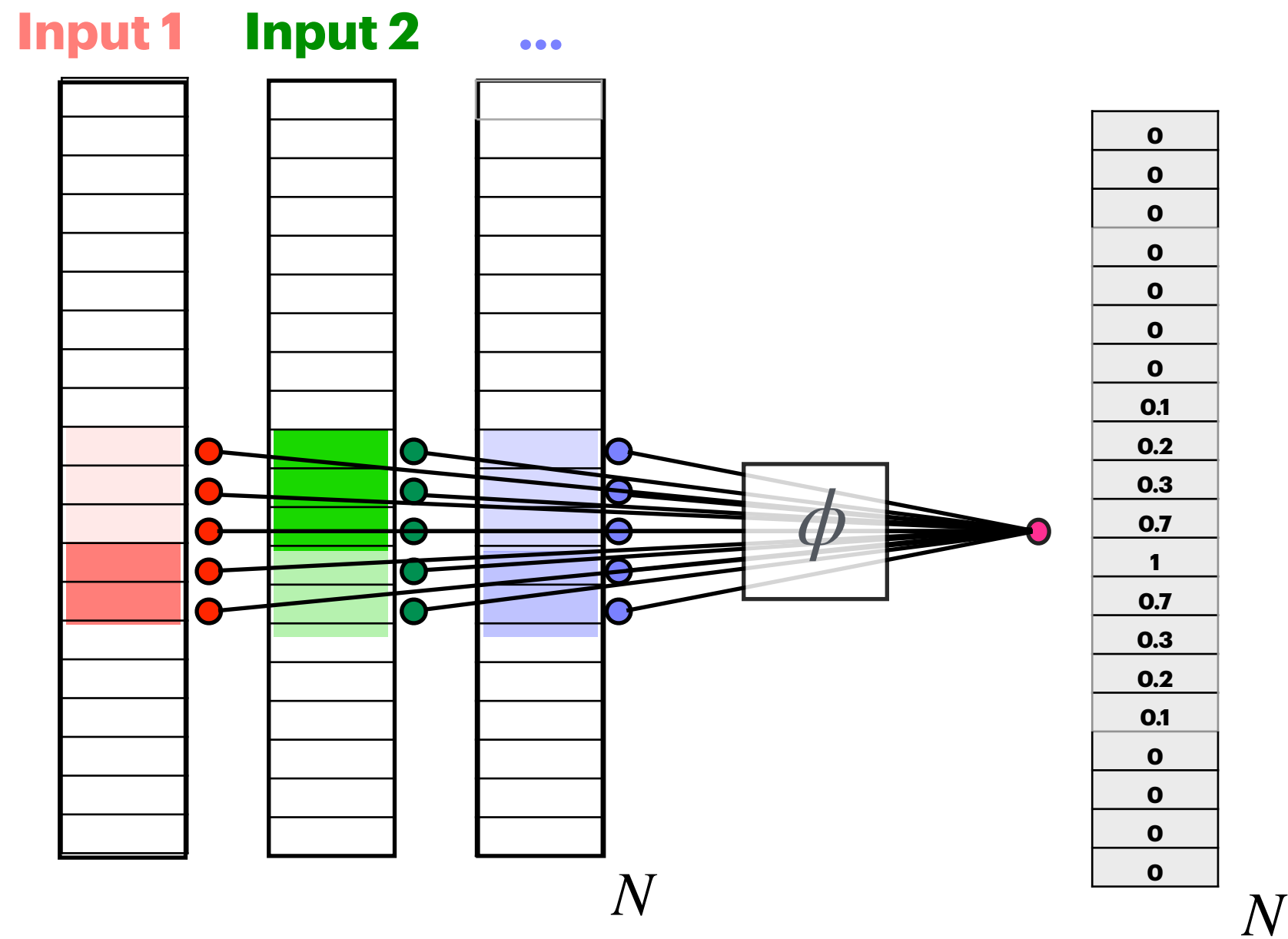
Questions: For a 1D input $x \in \mathbb{R}^N$, compare: Compare one dense linear layer $\mathbb{R}^N \rightarrow \mathbb{R}^N$ with one 1D convolutional filter of width k . Which one uses fewer learnable parameters?



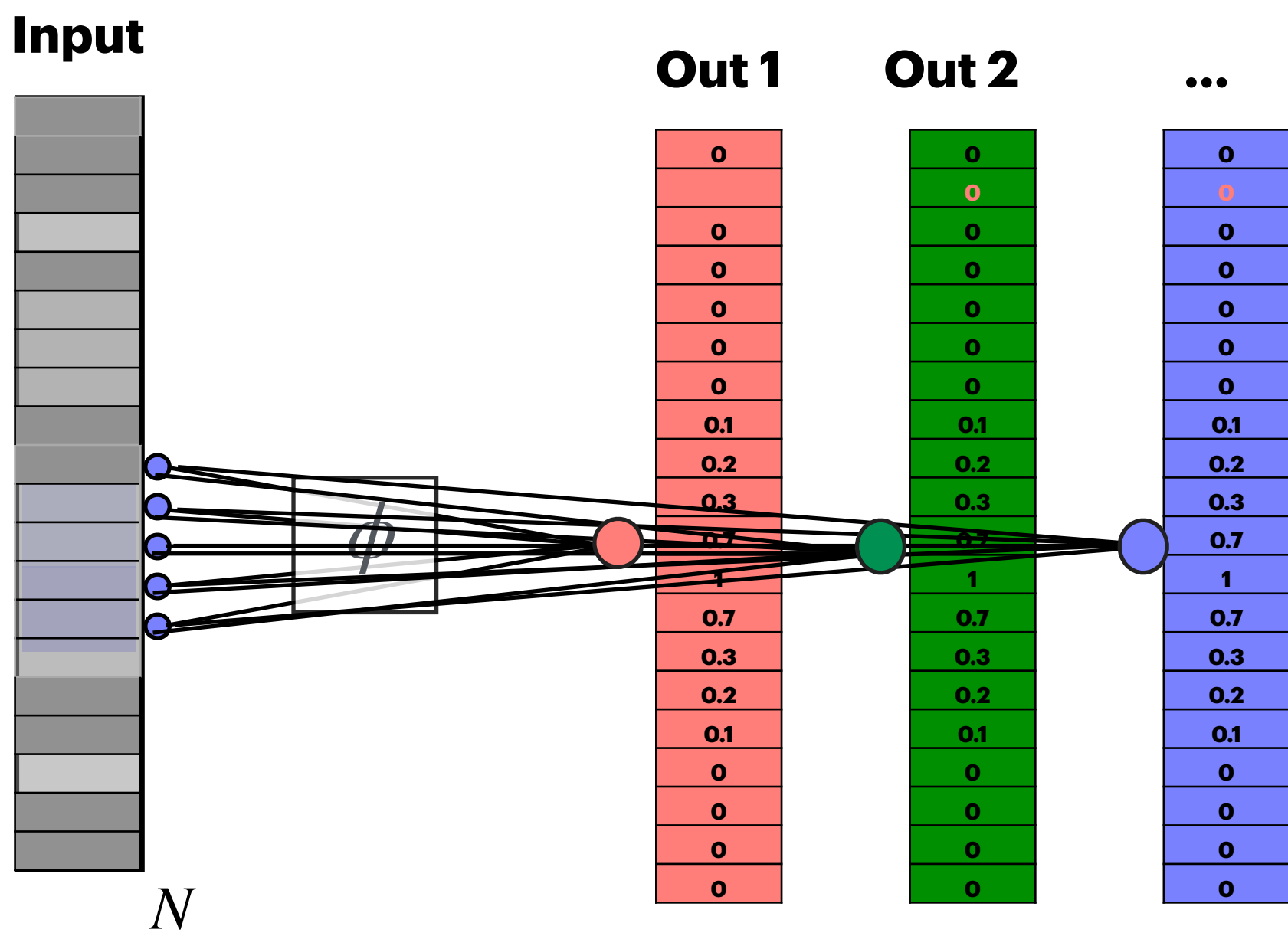
- MLP parameters = $N^2 + N$
- Convolutional Layer parameters: $k + 1$
- This is called **Weight Sharing**, which is an important concept

- What if we have color? Multiple input channels, or even multiple output channel?

Map multiple in channels to on out channel



Map single in channels to multiple out channel



To conclude:

$$F(x) = \phi \star x$$

was a Linear Convolutional Operator. Now it is time to introduce:

Concept 2: A Convolutional Block

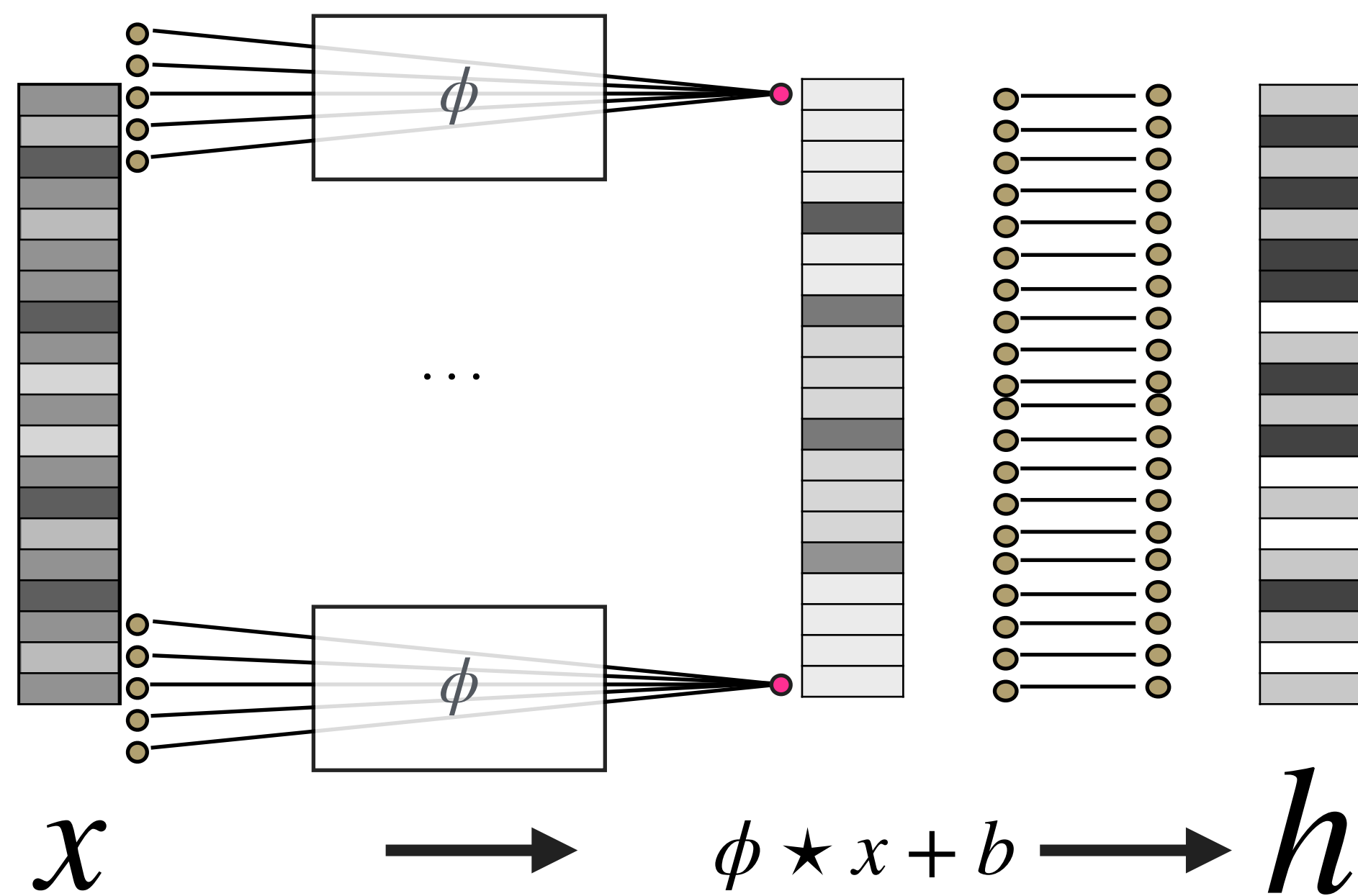
• **A convolutional Block** is built from:

a) the convolutional operator $F(x)$, followed by bias (this is usually called a convolutional Layer)

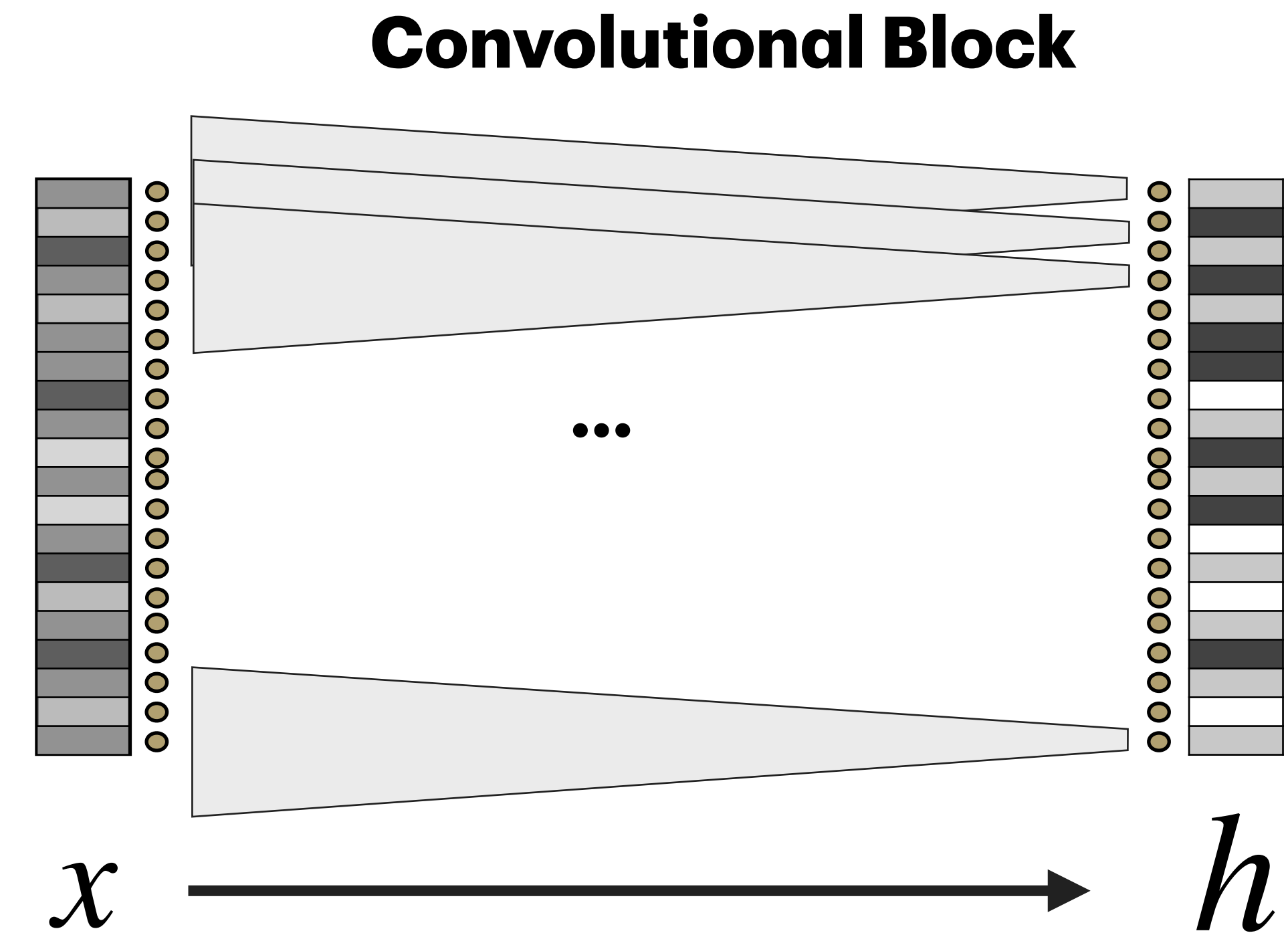
$$\phi \star x + b = F(x) + b$$

b) and then usually a nonlinearity σ , (usually ReLU):

$$h = \sigma(\phi \star x + b) = \sigma(F(x) + b)$$



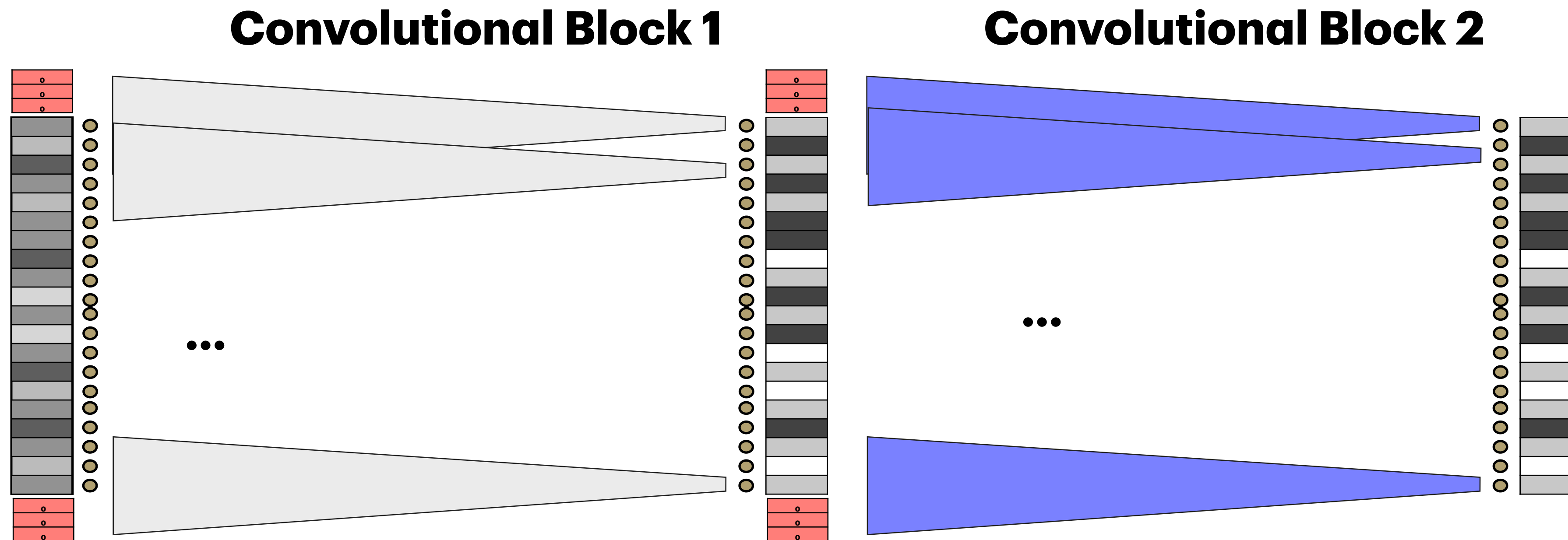
Or simply:



!! Note that the weights (w inside ϕ) and biases (b) are learnable via backprop

- **A note about Zero Padding:**

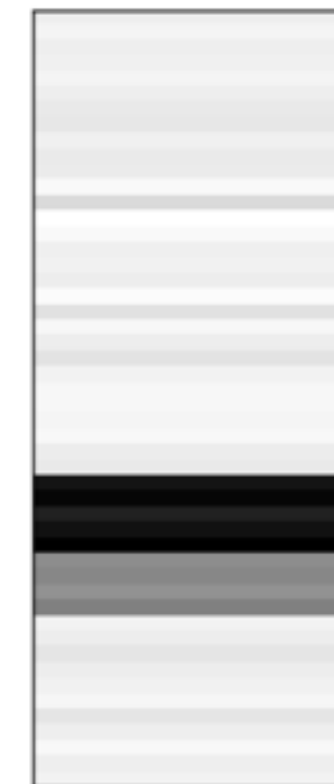
Padding adds **extra values (usually zeros)** around the input before applying the kernel. We often use it to keep the output grid the same size as the input grid.



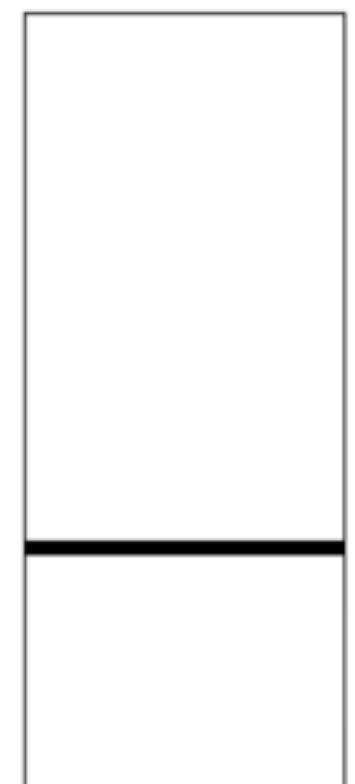
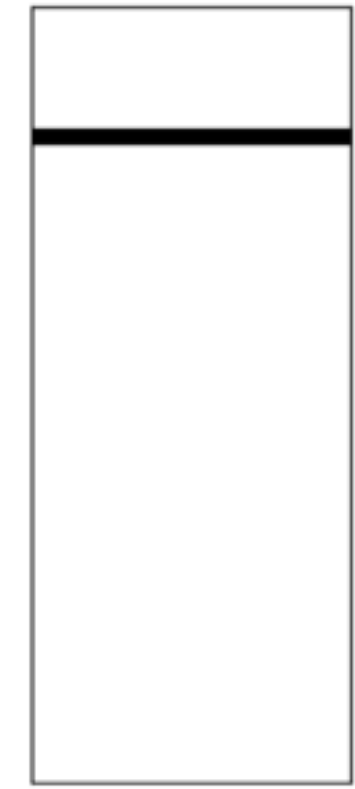
|| Pause Point: **Time to practice**

- We built a simple supervised 1D grid dataset for you:
 - each input contains one local “tree” pattern,
 - and the target is its center-response map.
- Now we define and train the simplest possible CNN: one convolutional layer with a single learnable kernel.
- Then we inspect two things: can it predict the center, and what pattern does the learned kernel capture?

Input: $x_i \in X$



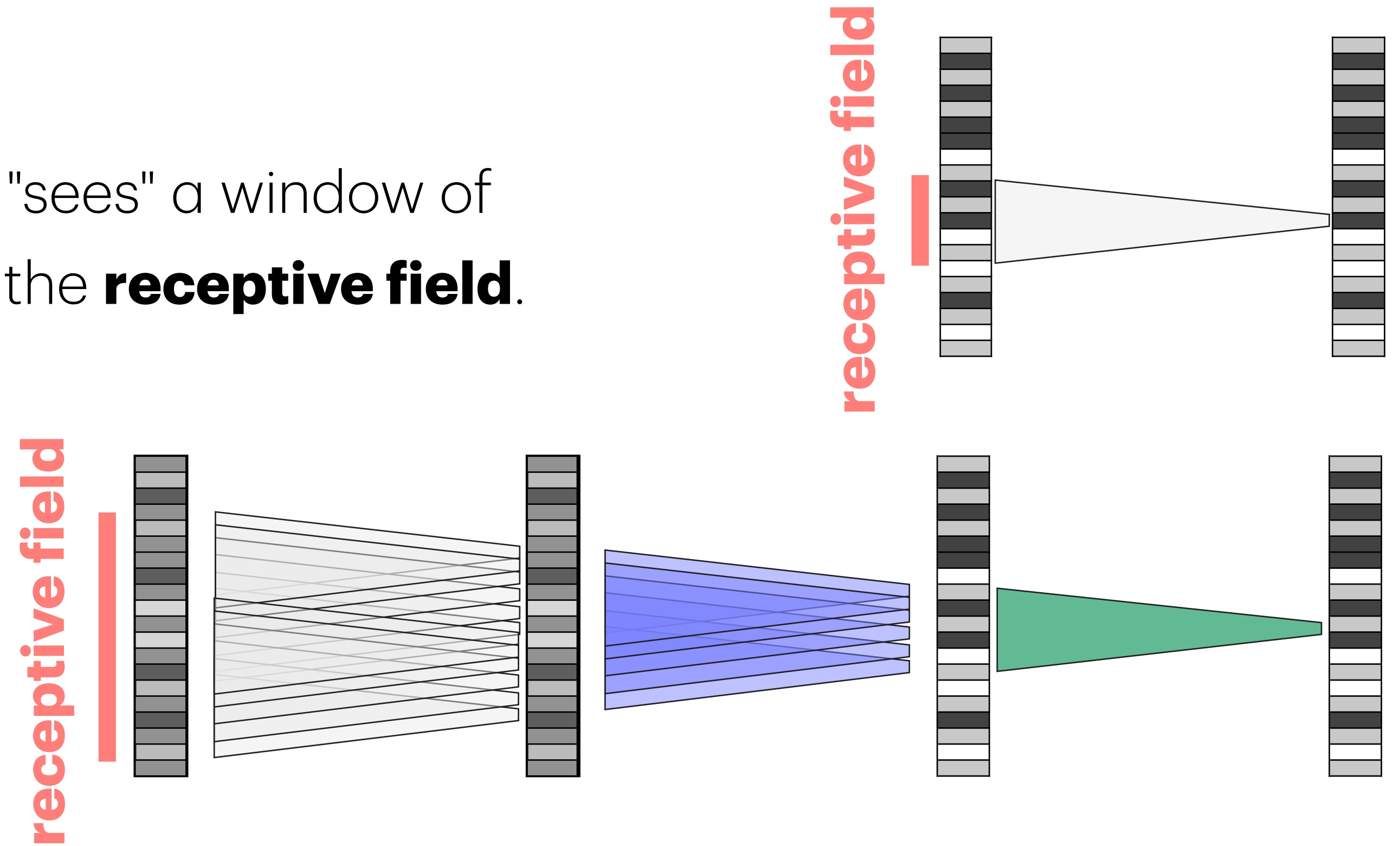
Target: $y_i \in Y$



Concept 3: The receptive field:

- Each output position in a **Conv Block** only "sees" a window of size k from the input. This window is called the **receptive field**.

- When we stack Conv Blocks, the receptive field of deeper layers grows.



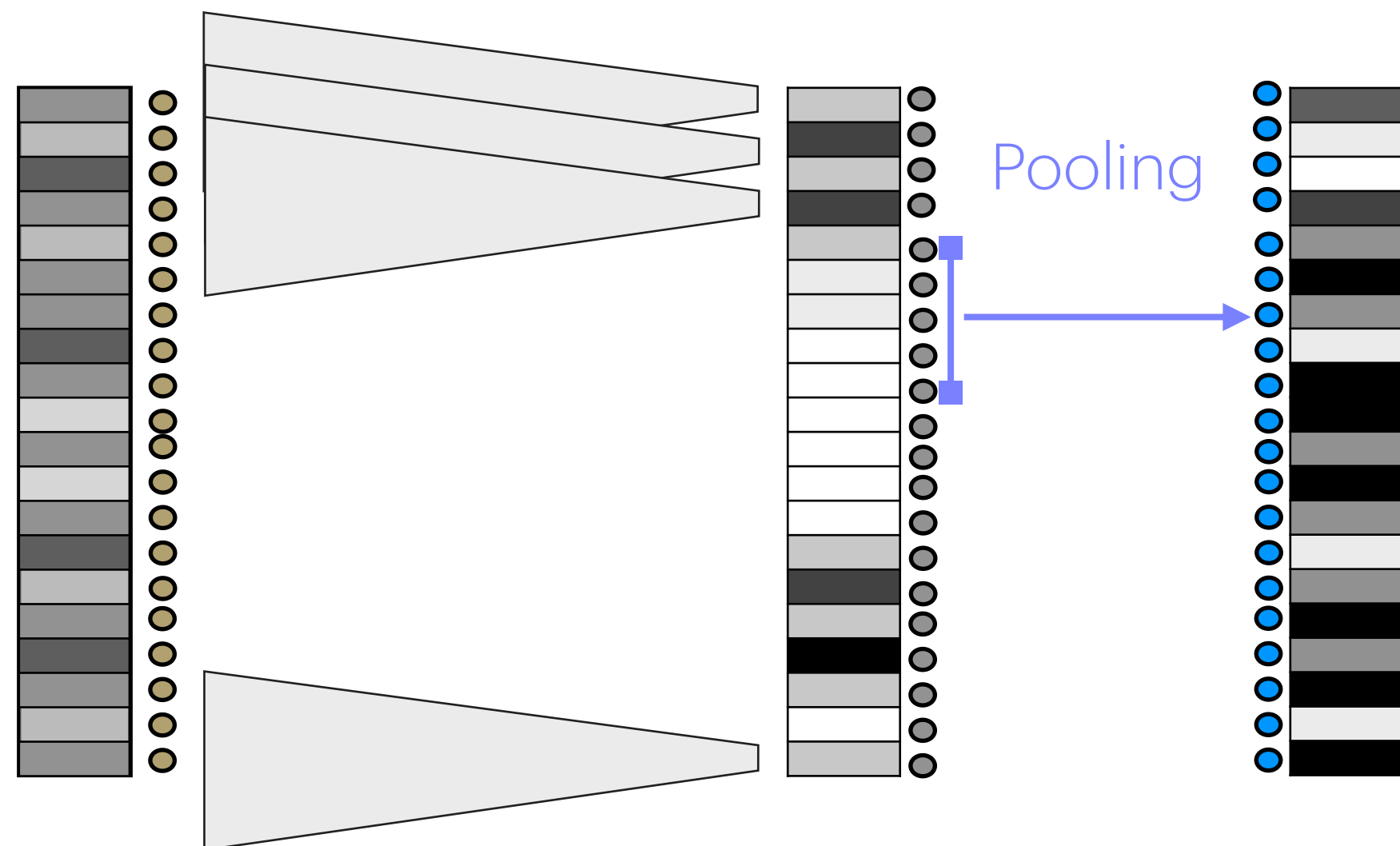
- There are two ways to control the receptive field size:
 - Kernel size k : a larger kernel sees more of the input per layer directly
 - Depth**: stacking more layers grows the receptive field indirectly, while keeping each individual kernel small
- In practice CNNs prefer small k (e.g. 3) and more depth: this gives the same receptive field as a large kernel, with fewer parameters and more nonlinearities in between, which increases expressive power.

Concept 4: Downsampling (via pooling and stride)

- After one or several conv blocks, we often apply a pooling operation.
- It reduces the spatial size of the feature map by summarizing local regions into single values. The most common type is **max pooling**. it take a window of size p and keep only the maximum value. The output is p times smaller than the input.
- Pooling also contributes a degree of robustness...

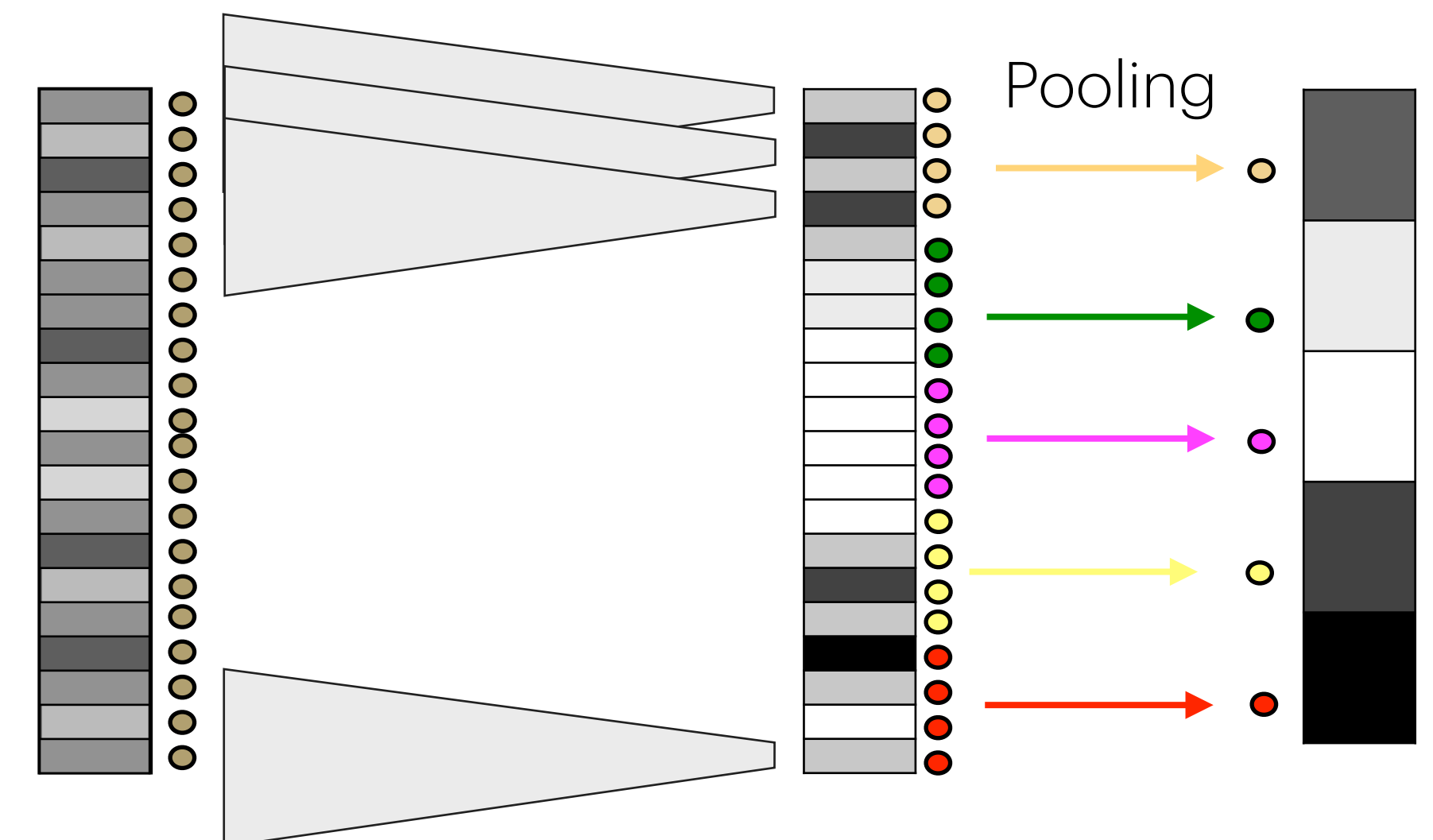
Pooling ($p=5$) with stride=1 (No downsampling)

Convolutional Block



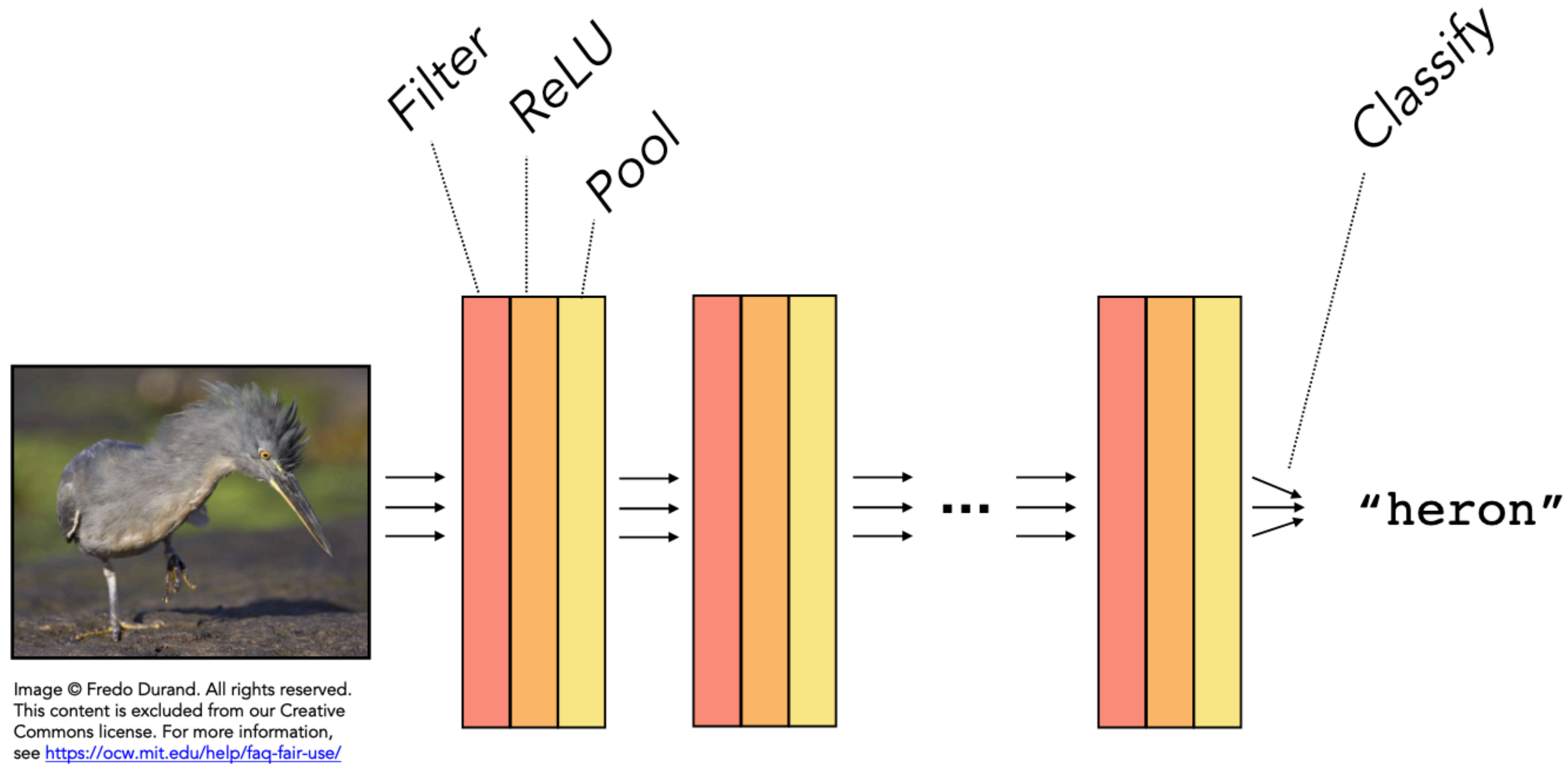
Pooling ($p=4$) with stride=4 (downsampling)

Convolutional Block

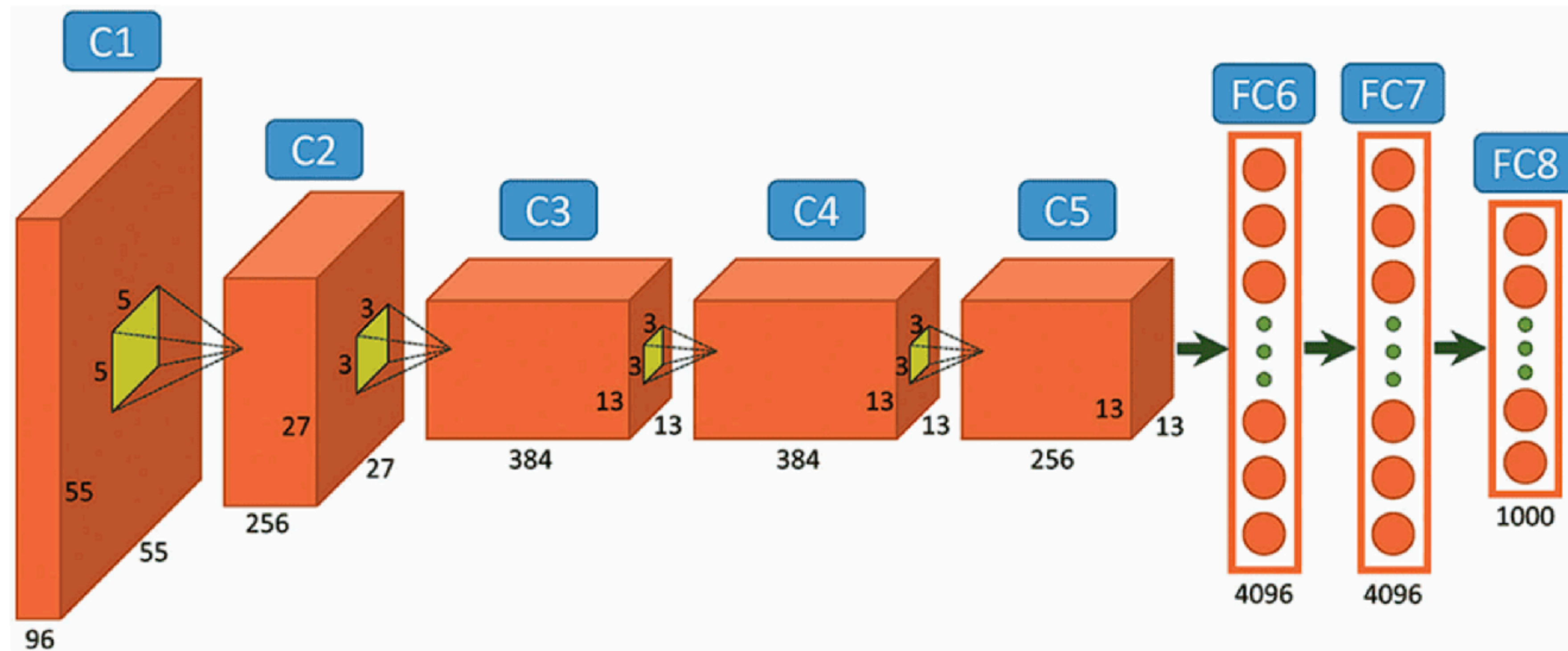


Popular CNN Architectures

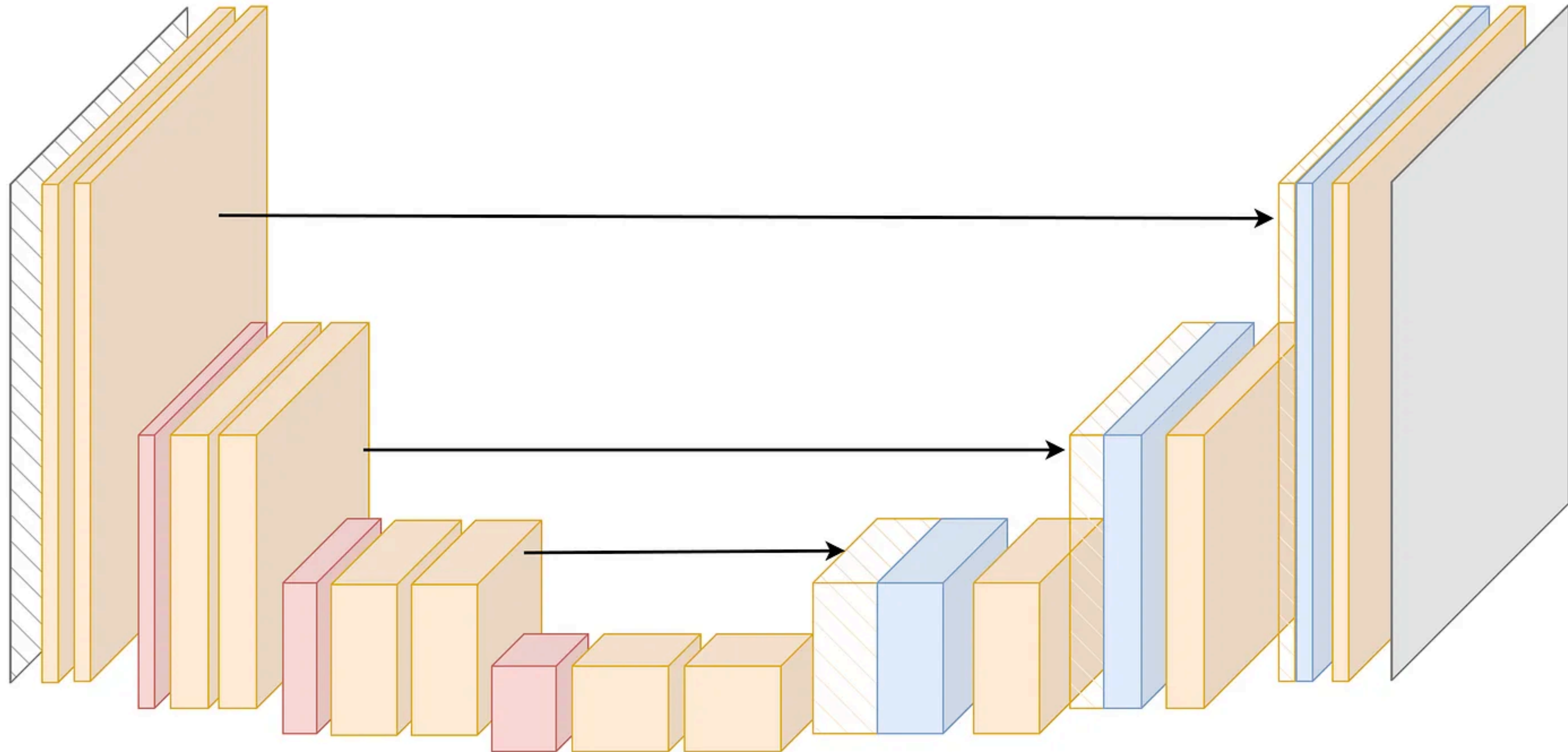
Computation in a neural net



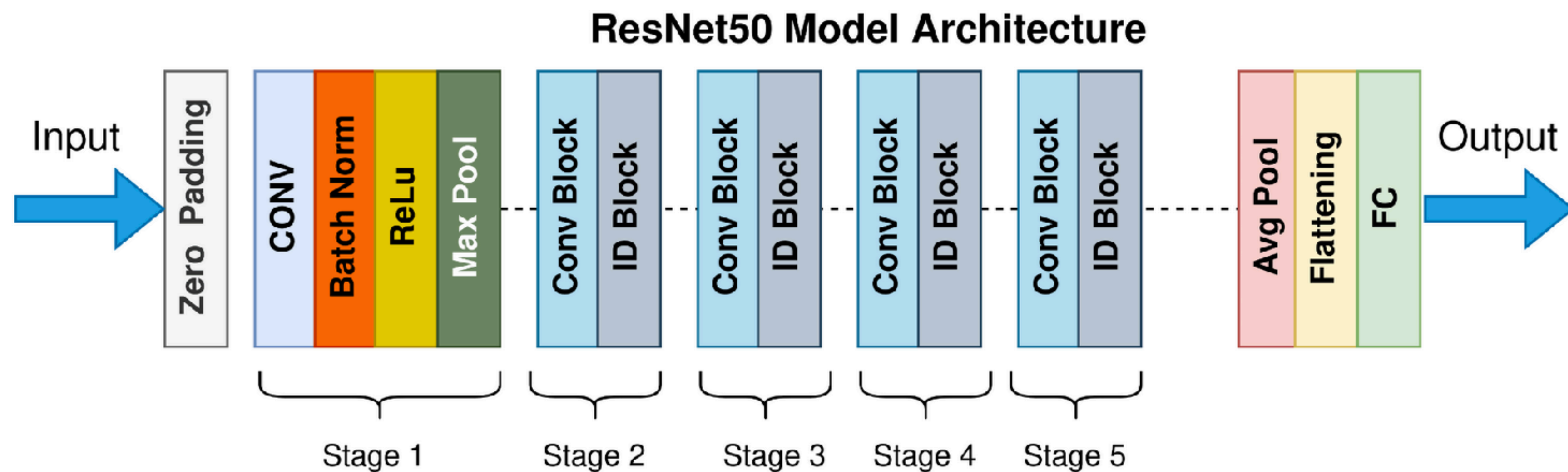
The AlexNet Architecture



U-Net



<https://medium.com/data-science/u-net-explained-understanding-its-image-segmentation-architecture-56e4842e313a>



Resnet-50 Model architecture

Read more

A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks., in: Nips, 2012: pp. 1097–1105.

K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition., in: Cvpr, 2016: pp. 770–778.

https://visionbook.mit.edu/convolutional_neural_nets.html